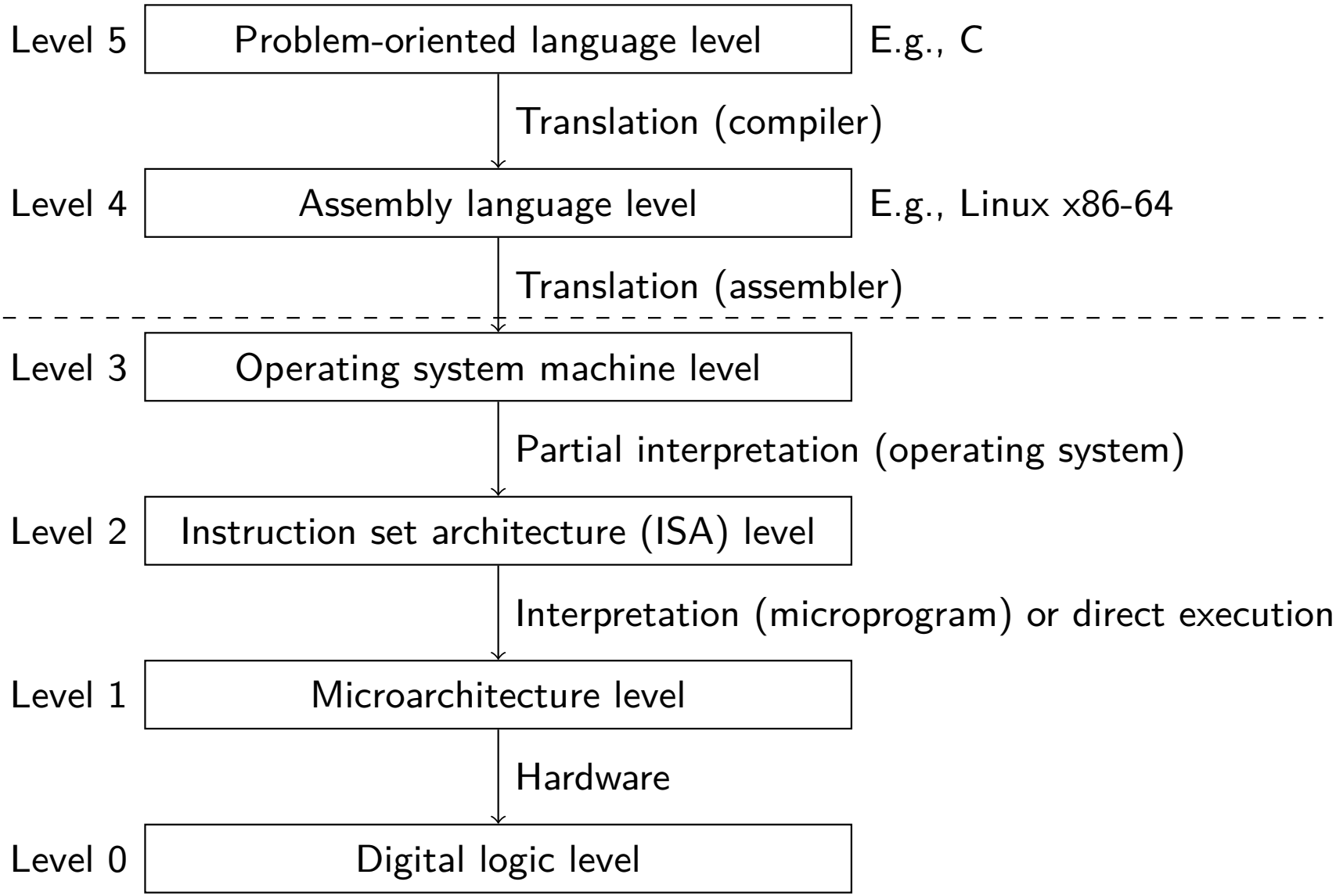
DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2026

Instruction Sets and x86-64 Assembly

- Introduction
- Where are we going with this?
- x86-64 Data Types and Register Organisation
- Instructions Basics
- Programs, Libraries, and the Operating System
- Putting It Together
- Addressing Memory
- Stack and Function Calls

Where Are We?


Level 5
Problem-oriented language level
E.g., C
Translation (compiler)
Level 4
Assembly language level
E.g., Linux x86-64
Translation (assembler)
Level 3
Operating system machine level
Partial interpretation (operating system)
Level 2
Instruction set architecture (ISA) level
Interpretation (microprogram) or direct execution
Level 1
Microarchitecture level
Hardware
Level 0
Digital logic level

Introduction

Aim: to gain introductory knowledge about

- characteristics of machine instructions in general, and its representation in assembly,
- how higher-level programming constructs can be implemented in assembly,
- Linux x86-64 assembly in detail.

Introduction

Aim: to gain introductory knowledge about

- characteristics of machine instructions in general, and its representation in assembly,
- how higher-level programming constructs can be implemented in assembly,
- Linux x86-64 assembly in detail.

Expectations for the future

- Develop your assembly programming skills in the lab sessions and at home.
- Use this material actively for reference while programming. This include slides, notes, and the additional and supplemental material on itslearning.
- Use yours skills in the first project.
- This material does not describe all details, use it as a starting point to seek further information.

Terminology

- (Machine) instruction for a processor: an abstract description of some operation it can execute.
  For example: “add”, “move”, “multiply”.
- Instruction set for a processor: the collection of machine instructions it can execute.
- Machine code/language: the binary representation of machine instructions. The thing that is actually being executed.
- Assembly code/language: a human readable description of machine code, with additional facilities for programmers.
- Assembler: a program that converts assembly code to machine code. (actually to object code, but more later)

High-level Languages

Simplified we generally have:

- As many variables as we can name. Each typically has a type.
- Types: determines how data is interpreted.
- Types: determines which operations we can perform.
- Types: may contain internal structure (e.g., class members).
- Structuring constructs: if-then-else, for-loop, switch, functions.

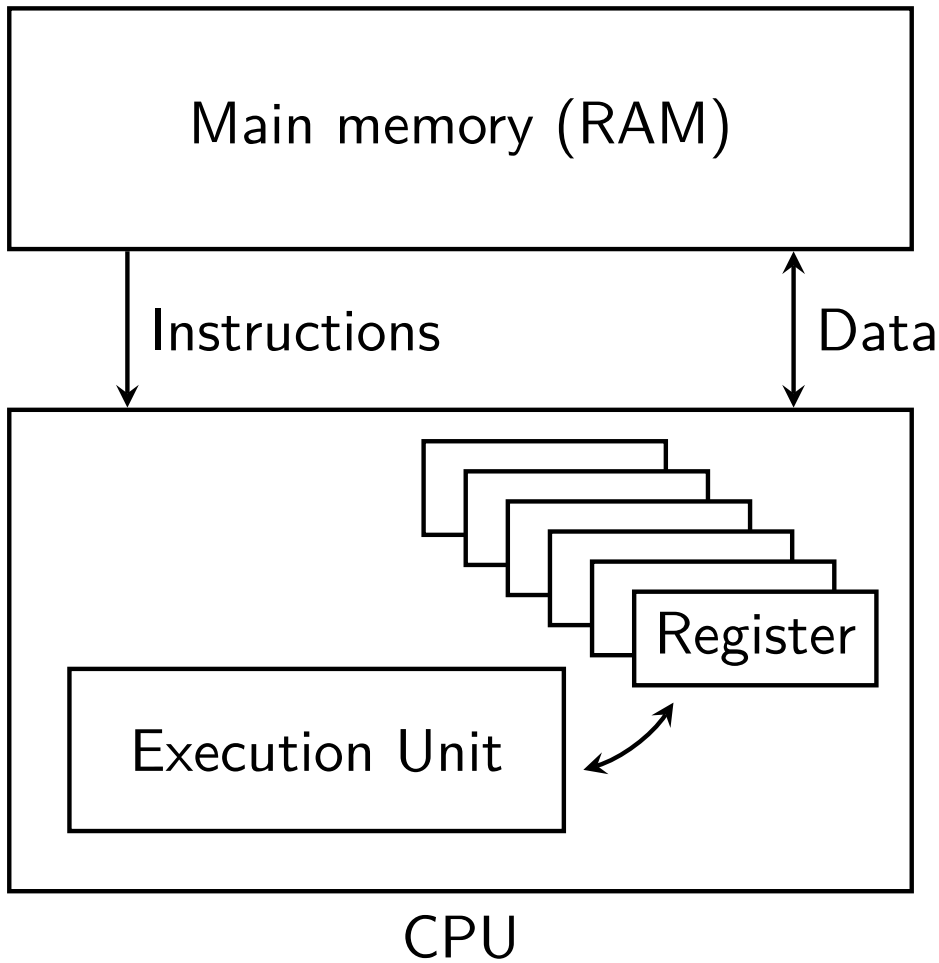
In assembly we basically have none of this!

Assembly

Generally:

- Instead of variables we have a limited pre-named set of registers.
- We also have the main memory, which we must manage explicitly.
- Registers have no (explicit) type, it's just raw bits.
  Main memory definitely has no type.
- How we use registers and memory is what determines what they mean.
- We must mentally kept track of what our data mean and how we have structured it.
- We must manually set up control flow with labels and jumps.
- Functions are implemented by establishing conventions.
  Remember, no variables so no formal function arguments.

A Very, Very, . . . , Very Simplified View of Our Computer



What is an "instruction"?

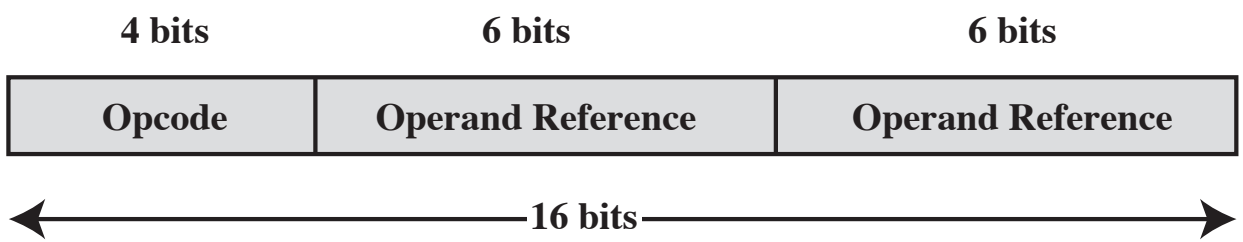
Recall the von Neumann Architecture: code is data.

Simplified, an instruction can have:

- Operation code (opcode): specifies what operation to perform. E.g., "add", "move", "jump".
- Source operand(s): a description of what data should be read.
- Result/target operand: where data, if any, should be written.
- Next instruction: where to fetch the next instruction.

All this is encoded as raw bits.

Example representation of an instruction:



What is an "instruction"?

Common operand types:

- Register: (almost) all CPUs has one or more registers.
- Main memory: a memory address.
- Immediate: a constant value, stored in the instruction it self.
- I/O device: an I/O module/device/port. With memory mapped I/O, this reduces to normal main memory operands.

Common instruction types:

- Arithmetic instructions: addition, division, ...
- Logic instructions: boolean logic, shifting, ...
- Data movement: to/from registers/memory
- Control flow: test, branch

Number of Operands

- Traditionally architectures can be classified by number of operands per instruction.
- This is less significant today, CPUs are anyway very complicated.

Different programs for calculating $Y = \frac{A-B}{C+D \cdot E}$?

Instruction		Comment
SUB	Y, A, B	Y ← A – B
MPY	T, D, E	T ← D × E
ADD	T, T, C	T ← T + C
DIV	Y, Y, T	Y ← Y ÷ T

(a) Three-address instructions

Instruction		Comment
MOVE	Y, A	Y ← A
SUB	Y, B	Y ← Y – B
MOVE	T, D	T ← D
MPY	T, E	T ← T × E
ADD	T, C	T ← T + C
DIV	Y, T	Y ← Y ÷ T

(b) Two-address instructions

Instruction		Comment
LOAD	D	AC ← D
MPY	E	AC ← AC × E
ADD	C	AC ← AC + C
STOR	Y	Y ← AC
LOAD	A	AC ← A
SUB	B	AC ← AC – B
DIV	Y	AC ← AC ÷ Y
STOR	Y	Y ← AC

(c) One-address instructions

Instruction Set Design

- Operation repertoire: how many and which operations to provide, and how complex operations may be.
- Data types: the various types of data upon which operations are performed.
- Instruction format: instruction length (in bits), number of addresses, size of various fields, ...
- Registers: number of registers that can be referenced by instructions, and how they can be used.
- Addressing: in which modes addresses in operands can be specified.

- Introduction
- Where are we going with this?
- x86-64 Data Types and Register Organisation
- Instructions Basics
- Programs, Libraries, and the Operating System
- Putting It Together
- Addressing Memory
- Stack and Function Calls

Hello World!

Linux x86-64 assembly, in AT&T syntax:

```
.section .data
hello: .ascii "Hello World!\n"

.section .text
.globl _start

_start:
    movq $1, %rax       # call the 'write' system call
    movq $1, %rdi       # write to file 1 (stdout)
    movq $hello, %rsi   # write data starting from the address
                        # that 'hello' points to
    movq $13, %rdx      # write 13 bytes
    syscall             # actually make the call

    movq $60, %rax      # call the 'exit' system call
    movq $0, %rdi       # return 0 as the exit code
    syscall             # actually make the call
```

```
> as hello.s -o hello.o # assembly -> object code
> ld hello.o -o hello # object code -> executable file
> ./hello
Hello World!
```

Example

High-level:

```
2     int rax = 0;
3     int rcx = 0;
4     while(rcx < 10) {
5        rax += rcx;
6        rcx += 1;
7     }
```

Assembly:

```
4     movq $0, %rax    # int rax = 0;
5     movq $0, %rcx    # int rcx = 0;
6  loopStart:          # {
7     cmpq $10, %rcx   #          rcx < 10
8     jge loopEnd
9     addq %rcx, %rax  #      rax += rcx;
10    addq $1, %rcx    #          rcx += 1
11    jmp loopStart    # }
12 loopEnd:
```

A label is not an instruction, but a symbolic name for a specific address in the running program.

- Introduction
- Where are we going with this?
- x86-64 Data Types and Register Organisation
- Instructions Basics
- Programs, Libraries, and the Operating System
- Putting It Together
- Addressing Memory
- Stack and Function Calls

Data Types

Remember, these are all implied by which instructions and register references we use.

Widths (in bits):

- Byte: 8
- Word: 16
- Doubleword: 32
- Quadword: 64
- Double quadword: 128

Types:

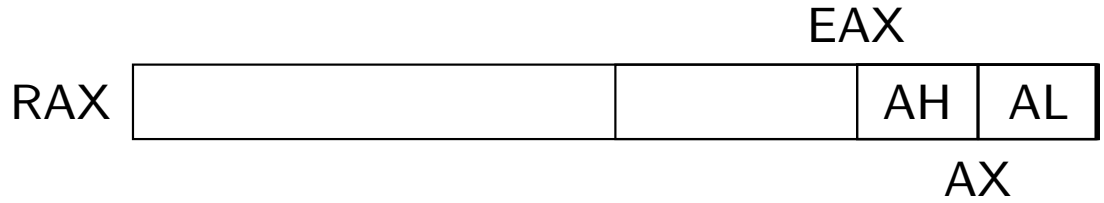
- Integer (signed and unsigned depends on specific instructions)
- Float

Text is just a sequence of 8-bit integers (see `man ascii`).

Registers (some of them)

Name	(Historic) Meaning
	General Purpose (64 bit)
RAX	Accumulator for arithmetic operations.
RBX	Base, holding a pointer to data.
RCX	Counter used in shift/rotate instructions and loops.
RDX	Data, used in arithmetic operations and I/O operations.
RSI	Source index, a pointer to data source in stream operations.
RDI	Destination index, a pointer to data destination.
R8–R15	New registers introduced in 64-bit mode.
RSP	Stack pointer, pointer to the top of the stack.
RBP	Stack frame base, pointer to base of current stack frame.
	Other
RFLAGS	Status bits, set and read by instructions.
FS, GS, ...	Segment registers.
RIP	Instruction pointer.

Multiple Names, Different Parts



- RAX: 64 bit
- EAX: the lower 32 bit of RAX
- AX: the lower 16 bit of EAX
- AH: the upper 8 bits of AX.
- AL: the lower 8 bits of AX.

Correspondingly for RBX, RCX, and RDX.
Also for RSI, RDI, RSP, and RBP, but no 8-bit versions.
For R8–R15:

- R*n*: 64 bit
- R*n*D: lower 32 bit ("doubleword")
- R*n*W: lower 16 bit ("word")
- R*n*L: lower 8 bit
- No equivalent to *x*H.

Why so complicated? history, and backwards compatibility

- Introduction
- Where are we going with this?
- x86-64 Data Types and Register Organisation
- **Instructions Basics**
- Programs, Libraries, and the Operating System
- Putting It Together
- Addressing Memory
- Stack and Function Calls

Assembly Code: x86-64 in AT&T Syntax

Opcodes are written symbolically with mnemonics. Examples:

- the move instruction is written "`mov`"
- the compare instruction is written "`cmp`"

We use the GNU Assembler (`as`).
Two common syntaxes: Intel and AT&T. We use AT&T.
General form of an instruction:

$$\langle \text{op} \rangle \ \langle \text{source} \rangle, \langle \text{target} \rangle$$

Most instructions (can) have a width specification:

- b: byte (8 bit)
- s: short (16-bit integer) or single (32-bit floating point)
- w: word (16 bit)
- l: long (32-bit integer or 64-bit floating point)
- q: quad (64 bit)

Example: "`movq`" for "move 64 bits", "`movb`" for "move 8 bits".

More (AT&T) Syntax

- Registers are written as "`%`⟨name⟩", usually in lower case, e.g.:
  `%rax`
- Comments start with "`#`" (until end of line):
  `# This is an awesome instruction`
- Immediate values start with "`$`" (or "`$0x`" for hex):
  `$256` (or `$0x100`)

Instruction Examples

- `movq $5, %rax`
  Move the value 5 to register RAX.
- `subq %rax, %rbx`
  RBX = RBX − RAX
  (there is also "`add`")
- `imulq %rcx`
  RDX:RAX = RAX · RCX (signed multiplication)
  (use "`mul`" for unsigned multiplication)
- `imulq $5, %r8`
  R8 = R8 · 5 (signed multiplication)
  (result is truncated to 64 bits)
- `idivq %rcx`
  RAX = RDX:RAX / RCX and RDX = RDX:RAX mod RCX
  Signed division (use "`div`" for unsigned)
- `cqto`
  RDX = sign extension of RAX
  (e.g., used before signed division and multiplication)

Example

- There is no assignment statement, only move instructions.

```
movq $42, %rax
addq $60, %rax
subq $5, %rax
movq $3, %rbx
cqto
idivq %rbx
# now: RAX = (42 + 60 - 5) / 3 = 32
# now: RDX = (42 + 60 - 5) mod 3 = 1
```

Control Flow

To jump around we must label the address of target instructions:

⟨labelName⟩:

- `cmpq $42, %rax`
  Perform "RAX − 42" and set flags from the result.
  (If comparing with an immediate, it must be on the left)
- `test %rbx, %r10`
  Perform "RBX bitwise-AND R10" and set flags from the result.
- `jmp myAdress`
  RIP = myAddress
  Jump to a different part of the program.
- `jg myAddress`
  Jump if greater, to myAddress.
  Uses flags to determine "if greater".
  Also: `je`, `jne`, `jge`, `jl`, `jle`, ...

Note: many more instructions modifies the flags, so be careful.

[https://en.wikibooks.org/wiki/X86_Assembly/Control_Flow]

Example

High-level:

```
2     int rax = 0;
3     int rcx = 0;
4     while(rcx < 10) {
5        rax += rcx;
6        rcx += 1;
7     }
```

Assembly:

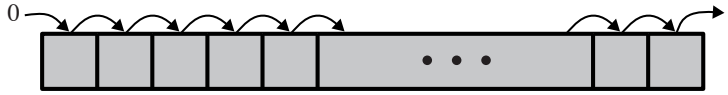
```
4     movq $0, %rax    # int rax = 0;
5     movq $0, %rcx    # int rcx = 0;
6  loopStart:          # {
7     cmpq $10, %rcx   #          rcx < 10
8     jge loopEnd
9     addq %rcx, %rax  #      rax += rcx;
10    addq $1, %rcx    #          rcx += 1
11    jmp loopStart    # }
12 loopEnd:
```

A label is not an instruction, but a symbolic name for a specific address in the running program.

More Instruction Examples

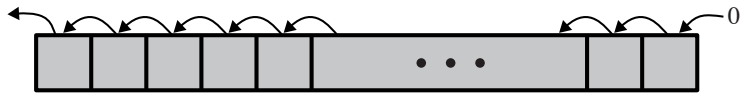
- `andq %rbx, %rax`
  RAX = RAX bitwise-AND RBX
  Also: `or`, `xor`
- `shl $8, %rsi`
  Logical left-shift of RSI by 8 places.
  Also: (see next slide)

Shift Instructions



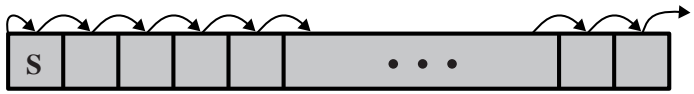
(a) Logical right shift

`shr`



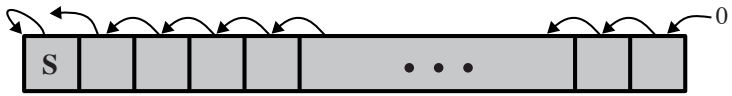
(b) Logical left shift

`shl`



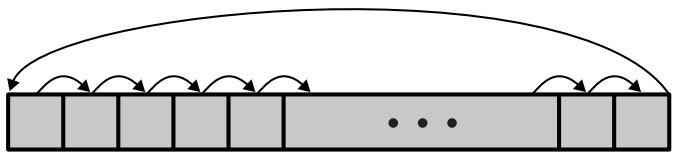
(c) Arithmetic right shift

`sar`



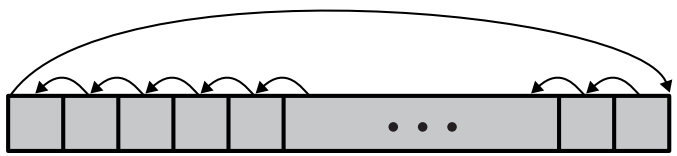
(d) Arithmetic left shift

`sal`



(e) Right rotate

`ror`



(f) Left rotate

`rol`

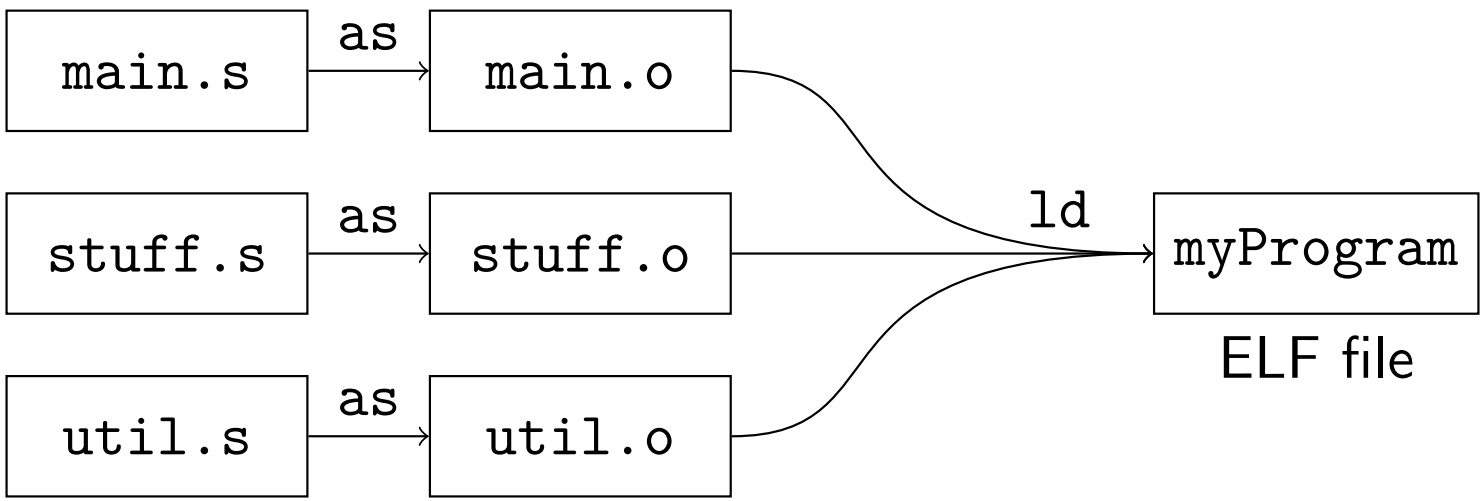
- Introduction
- Where are we going with this?
- x86-64 Data Types and Register Organisation
- Instructions Basics
- Programs, Libraries, and the Operating System
- Putting It Together
- Addressing Memory
- Stack and Function Calls

What is an Executable Program?

- It must have some machine code.
  E.g., translated from our assembly code.
- It probably also needs some space for (read-only) data.
- It must indicate where to start execution.
  I.e., we must label the first instruction in the program.
- Almost all programs we use call code in libraries.
- A library is also "just" a bunch of machine code and data.
- There are many different file formats for executables/libraries.
- On Linux the common format is ELF
  (Executable and Linkable Format).

What is an Executable Program?

- An ELF file has sections for different types of data, e.g.:
  - ".text": machine code.
  - ".data": read-write data.
  - ".rodata": read-only data.
- Simplified:
  - Assembly code is a textual description of how to create those sections.
  - The assembler lets us create a fragment of an ELF file, called an object file.
  - A linker is a program that takes object files and makes an ELF file. It will also resolve symbols mentioned in different files.
  - A symbol is a label with some meta-information.



Assembler Directives

- We must tell the assembler what to put into object files.
- We have already seen the instructions.
- An assembler directive starts with a dot:
  "`.`⟨directiveName⟩"
- Switching section: "`.section` ⟨sectionName⟩"
  E.g., "`.section .text`"
  means "The following is executable code."
  and "`.section .data`"
  means "The following is space for non-executable data."
- Actually making space for data: "`.space` ⟨numBytes⟩"
- Inserting text data: "`.ascii "`⟨someText⟩`"`"
  See also "`.byte`", "`.quad`", ... for inserting integers.

We probably want to label the space/data we insert, so we can refer to it later, e.g.,:

```
myString: .ascii "Hello World!\n"
myInt:    .quad 42
```

Symbols

Simplified, a library has:

- Some code, with a label for each callable function.
- Some data, with a label for each accessible variable.
- A symbol table that indexes the functions and variables.

An executable program in addition has:

- A special accessible symbol called "`_start`" for where to start execution.

An externally accessible symbol is called a global symbol.
To make a symbol global, use "`.globl` ⟨symbolName⟩"

Commands to inspect object/library/executable files: **`nm`**, **`objdump`**, **`readelf`**

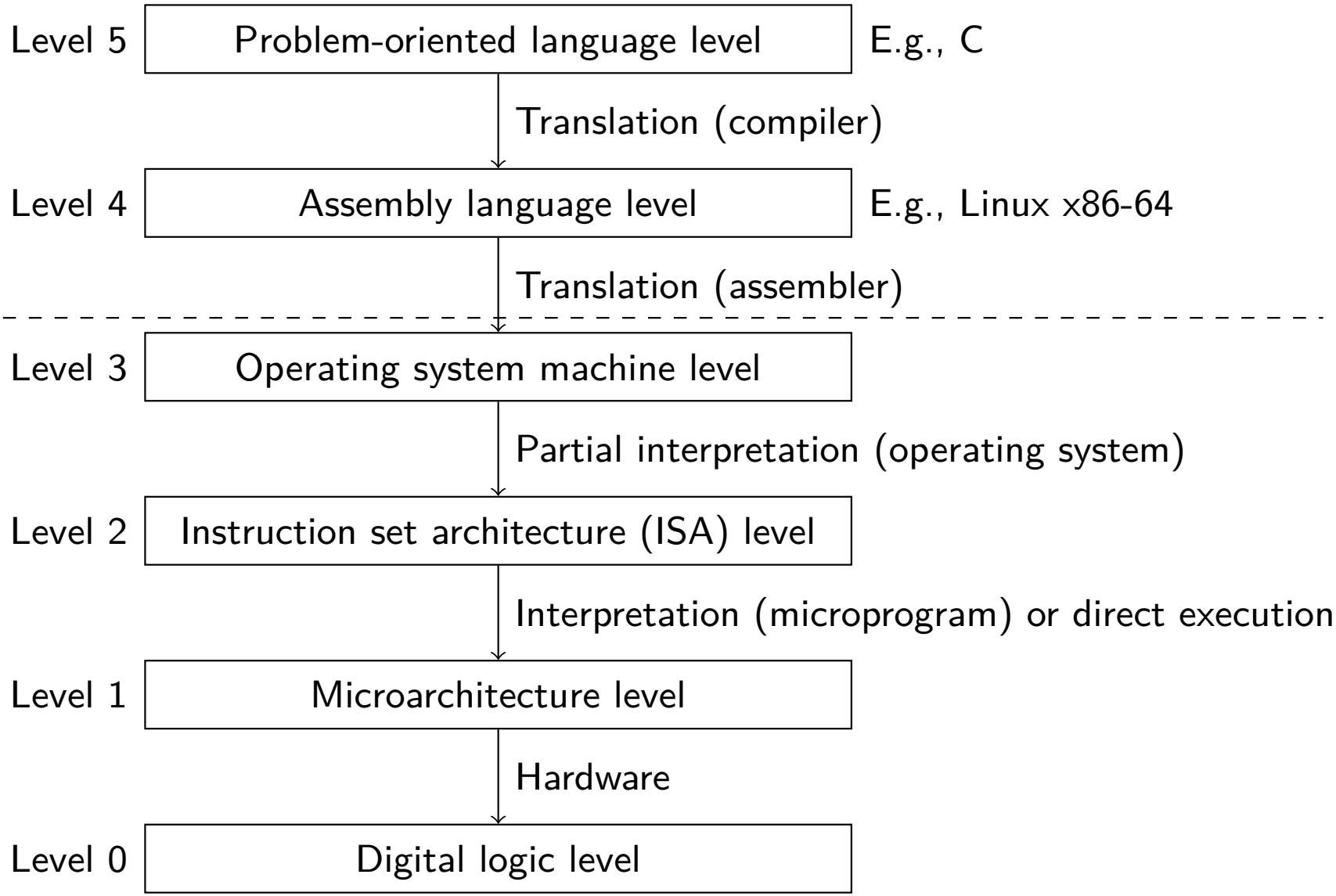
```
> nm hello.o # show symbol table
> objdump -d hello # show disassembled code
> objdump -s -j .data # show contents of .data
```

Example

```
1  .section .data
2  bound:
3      .quad 10            # int bound = 10
4
5  .section .text
6  .globl _start
7  _start:
8      movq $0, %rax       # int rax = 0;
9      movq $0, %rcx       # int rcx = 0;
10 loopStart:              # {
11     cmpq bound, %rcx    #           rcx < bound
12     jge loopEnd
13     addq %rcx, %rax     #       rax += rcx;
14     addq $1, %rcx       #           rcx += 1
15     jmp loopStart       # }
16 loopEnd:
```

Much more on memory references later.

Abstraction Levels



Operating System

- A program without side effects is rather boring.
- How do we read input and arguments?
- How do we write output?
- How do we even stop the program?
- What about the exit code of the program?

Operating System

- A program without side effects is rather boring.
- How do we read input and arguments?
- How do we write output?
- How do we even stop the program?
- What about the exit code of the program?

System calls: the interface of the OS

- `syscall`:
  Ask the operating system to do a specific operation.
  Operation number: RAX.
  Arguments: in registers RDI, RSI, RDX, RCX, R8, R9.
  Return value: in RAX.
  The kernel may change all those registers and R10 and R11.

E.g., program exit: RAX $= 60$, RDI $= \langle$exit code$\rangle$

Note: other operating systems may have different `syscall` conventions.

- Introduction
- Where are we going with this?
- x86-64 Data Types and Register Organisation
- Instructions Basics
- Programs, Libraries, and the Operating System
- Putting It Together
- Addressing Memory
- Stack and Function Calls

Hello World!

Linux x86-64 assembly, in AT&T syntax:

```
.section .data
hello: .ascii "Hello World!\n"

.section .text
.globl _start

_start:
    movq $1, %rax       # call the 'write' system call
    movq $1, %rdi       # write to file 1 (stdout)
    movq $hello, %rsi   # write data starting from the address
                        # that 'hello' points to
    movq $13, %rdx      # write 13 bytes
    syscall             # actually make the call

    movq $60, %rax      # call the 'exit' system call
    movq $0, %rdi       # return 0 as the exit code
    syscall             # actually make the call
```

```
> as hello.s -o hello.o # assembly -> object code
> ld hello.o -o hello # object code -> executable file
> ./hello
Hello World!
```

Additional Notes

- `movq hello, %rax`
  Move from address "hello".
- `movq $hello, %rax`
  Use the address "hello" as an immediate value.
- A file descriptor is an integer that the kernel knows what means.
  The "open" system call will open a file and return a file descriptor.
- Special file descriptors:
  - 0: stdin
  - 1: stdout
  - 2: stderr
- In the shell, the exit code of the last program is "`$?`"
  E.g., `echo $?`
- Exit code convention:
  - 0: "all good"
  - not 0: "an error happened"

Other System Calls

- `read`: read some bytes
  - RAX: 0
  - RDI: file descriptor (0 for stdin)
  - RSI: address (pointer) to input buffer
  - RDX: size of buffer
  - Returns (in RAX): number of bytes read.
- `open`: open a file
  - RAX: 2
  - RDI: pointer to filename string
  - RSI: flags, access mode and options (create file, append, ...)
    Read-only: 0, write-only: 1, read/write: 2
  - RDX: mode, permissions for new files
    If no new file is created the value doesn't matter.
  - Returns: the file descriptor
- `close`: close a file
  - RAX: 3
  - RDI: file descriptor

See also `https://filippo.io/linux-syscall-table/`

Control Flow: Unconditional Jump

```
    ...
    jmp exit
    ...

exit:
    movq $60, %rax
    movq $0, %rdi
    syscall
```

```
    ...
loopStart:
    ...
    jmp loopStart
    ...
```

- `jmp` unconditionally jumps to a given address.
- Remember: a label is just a symbolic name for an address.

Control Flow: Conditional Jumps

```
    ...
    cmpq %rax, %rbx
    je exit
    ...

exit:
    movq $60, %rax
    movq $0, %rdi
    syscall
```

- `cmpq <arg1> <arg2>` compares two values and sets bits in the flags register. (many other instructions sets flags as well)
- Conditional jumps use those flags:
  - `je`: if <arg2> $=$ <arg1>
  - `jne`: if <arg2> $\neq$ <arg1>
  - `jg`: if <arg2> $>$ <arg1>
  - `jge`: if <arg2> $\geq$ <arg1>
  - `jl`: if <arg2> $<$ <arg1>
  - `jle`: if <arg2> $\leq$ <arg1>

- Introduction
- Where are we going with this?
- x86-64 Data Types and Register Organisation
- Instructions Basics
- Programs, Libraries, and the Operating System
- Putting It Together
- Addressing Memory
- Stack and Function Calls

Addressing Modes

- The field(s) in a typical instruction format are relatively small, probably too small to contain an address to any memory location we are interested in.
- Different addressing modes can be used to specify addresses.
- Generally there is some trade-off between address range, flexibility, and/or complexity between the modes.

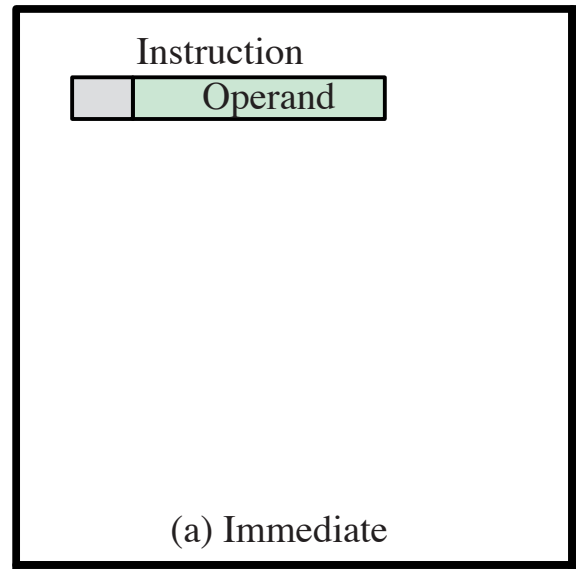
Addressing Modes

Typical Modes (not just x86-64):

- Immediate
- Direct
- Indirect
- Register
- Register indirect
- Displacement
- Stack

Addressing Modes: Immedidate

- Operand = A
- A constant value directly in the instruction.
- Not really “addressing”.
- No memory is referenced.
- Limited operand size.

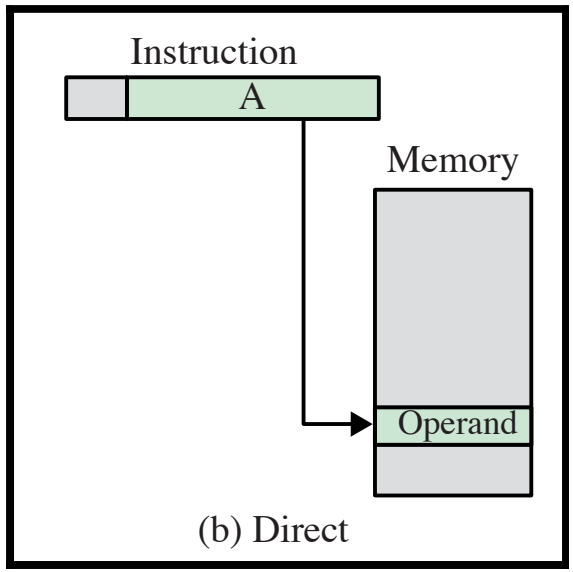


Notation:

- A: contents of an address field in the instruction
- R: contents of an address field in the instruction that refers to a register
- (X): contents of memory at location X or of register X
- EA: effective (actual) address

Addressing Modes: Direct

- EA = A
- Operand = (EA)
- Very common in older computers.
- Simple.
- Limited address range.

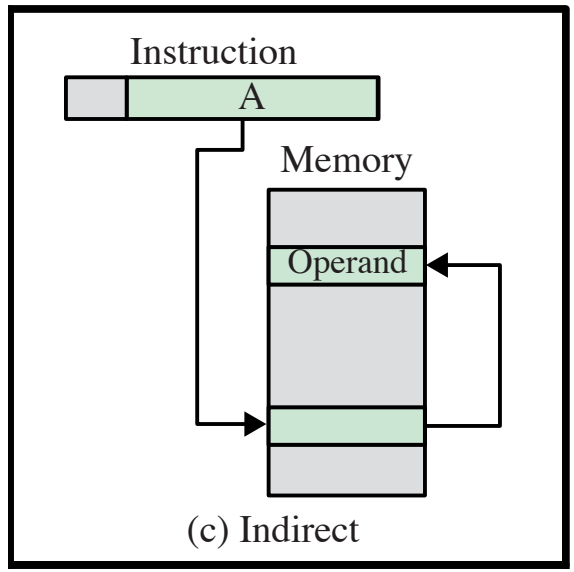


Notation:

- A: contents of an address field in the instruction
- R: contents of an address field in the instruction that refers to a register
- (X): contents of memory at location X or of register X
- EA: effective (actual) address

Addressing Modes: Indirect

- EA = (A)
- Operand = (EA)
- Use a shorter address to find a longer address.
- Large address range.
- Multiple memory references.

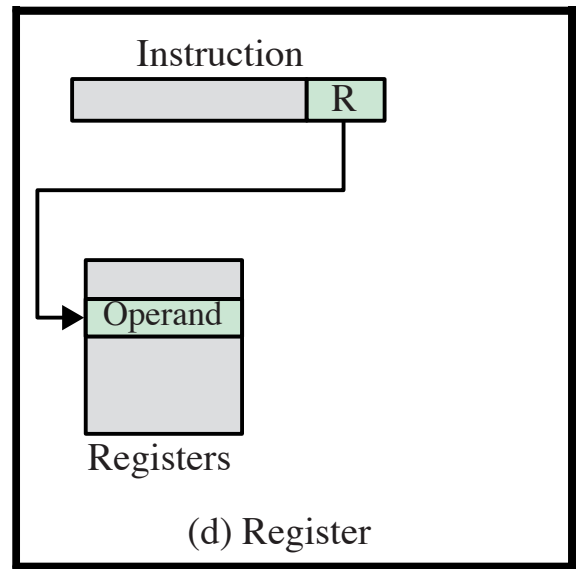


Notation:

- A: contents of an address field in the instruction
- R: contents of an address field in the instruction that refers to a register
- (X): contents of memory at location X or of register X
- EA: effective (actual) address

Addressing Modes: Register

- EA = R
- Operand = (R)
- Similar to direct addressing, but targeting the register bank.
- Extremely fast, no memory accesses.
- Limited "address" range.

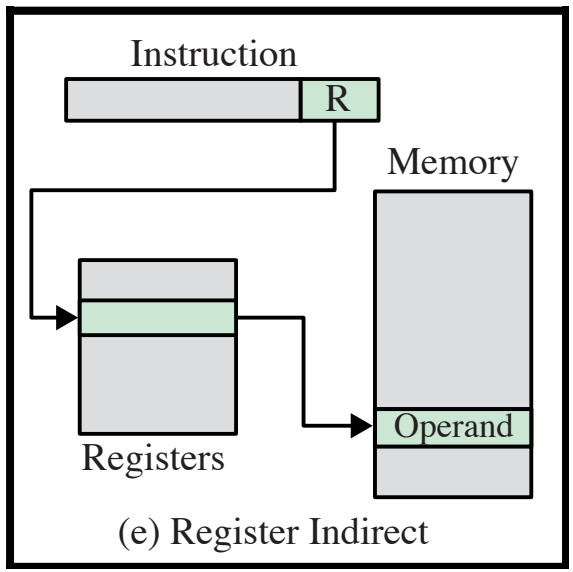


Notation:

- A: contents of an address field in the instruction
- R: contents of an address field in the instruction that refers to a register
- (X): contents of memory at location X or of register X
- EA: effective (actual) address

Addressing Modes: Register Indirect

- EA = (R)
- Operand = (EA)
- Similar to indirect addressing, but using a register as the intermediary.
- Large address space.
- Has a memory reference.

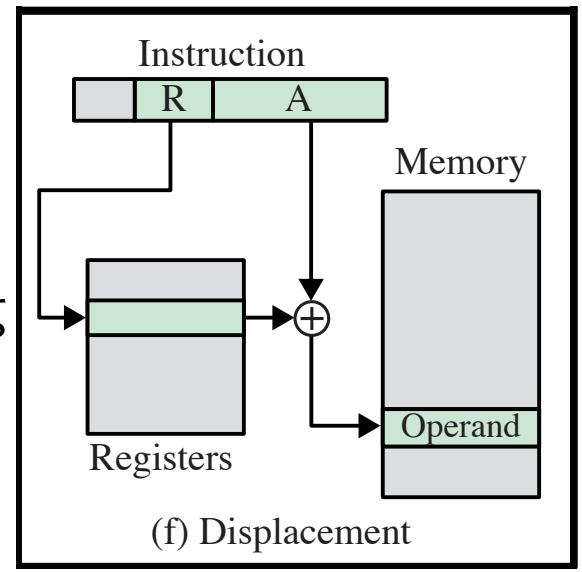


Notation:

- A: contents of an address field in the instruction
- R: contents of an address field in the instruction that refers to a register
- (X): contents of memory at location X or of register X
- EA: effective (actual) address

Addressing Modes: Displacement

- EA = A + (R)
- Operand = (EA)
- Very powerful but more complex.
- The instruction must have two addressing fields.
- There are multiple variations of the scheme.



Notation:

- A: contents of an address field in the instruction
- R: contents of an address field in the instruction that refers to a register
- (X): contents of memory at location X or of register X
- EA: effective (actual) address

Addressing Modes: Displacement

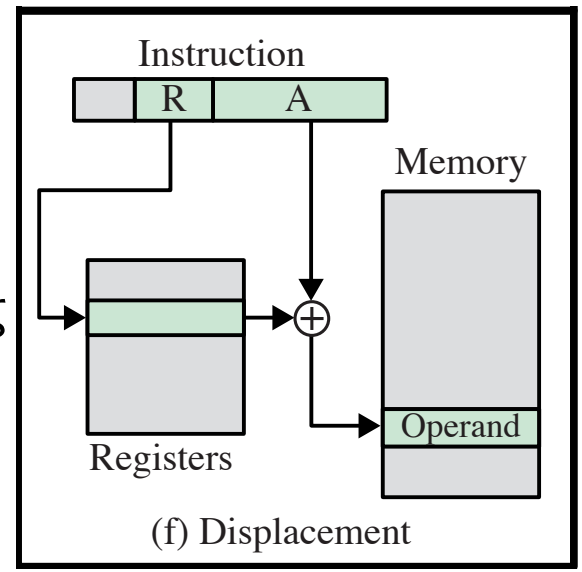
- Relative Addressing
  - Also called PC-relative addressing.
  - The PC (program counter, instruction pointer) is the base register.
  - The effective address is relative to the address of the instruction.
  - Exploits locality of references.
  - Saves addressing bits if most memory references are near.

Addressing Modes: Displacement

- Base-Register Addressing
  - A register contains an address, and another addressing field contains an offset.
  - The register may be explicitly specified in the instruction, or be implicit.
  - Exploits locality of references.
  - Convenient for segmentation of memory.
- Indexing
  - A register contains an offset, and an address field contains a memory address.
  - Almost the same as base-register addressing.
  - Important for iterative operations
    (e.g., iterating through arrays)

Addressing Modes: Displacement

- EA = A + (R)
- Operand = (EA)
- Very powerful but more complex.
- The instruction must have two addressing fields.
- There are multiple variations of the scheme.

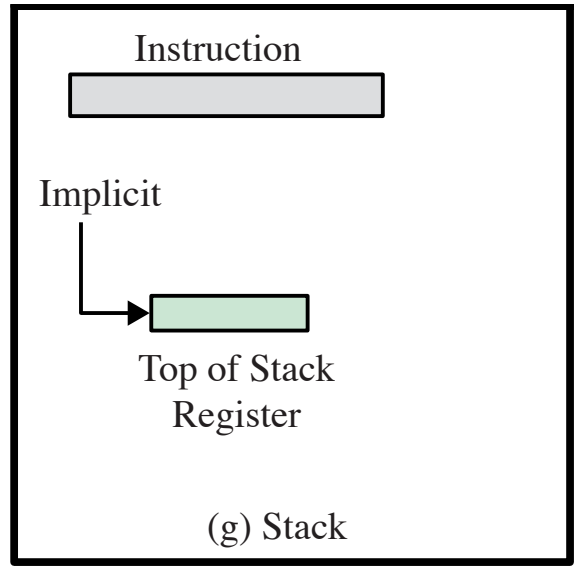


Notation:

- A: contents of an address field in the instruction
- R: contents of an address field in the instruction that refers to a register
- (X): contents of memory at location X or of register X
- EA: effective (actual) address

Addressing Modes: Stack

- EA = top of stack
- Operand = (EA)



Notation:

- A: contents of an address field in the instruction
- R: contents of an address field in the instruction that refers to a register
- (X): contents of memory at location X or of register X
- EA: effective (actual) address

Stack Addressing

- A stack is a linear array of locations.
- Sometimes called a "pushdown list" or "last-in-first-out queue".
- The x86-64 stack grows from high to low addresses.
- Most machines have a stack pointer register (e.g., RSP for x86-64).
- Therefore, stack addressing is a form of register indirect addressing, but with an implicit register.
- Many machines also have a pointer to the base of (part of) the stack (e.g., RBP).
- Instructions: `push`, `pop`.
- An essential feature for supporting functions (see later).

Overview

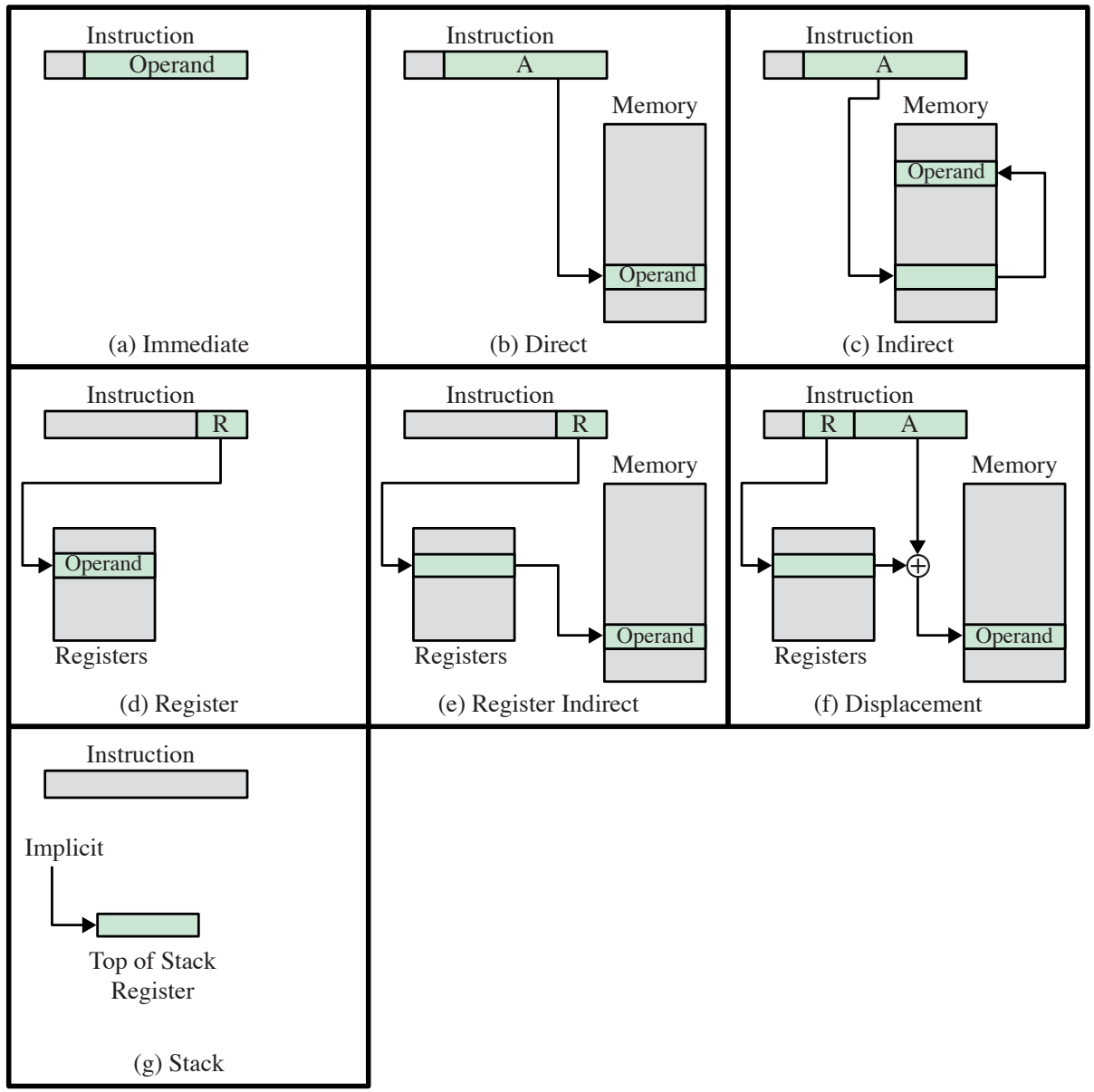


Figure 13.1 Addressing Modes

Virtual Addressing

- Most modern CPUs support virtual addressing, e.g., for paging.
- So the address calculation is even more complicated, with more indirection.
- However, this is transparent to the programmer.
- We just need to specify an effective address.
- The memory management unit (MMU) of the CPU calculates the actual address.

Addressing in x86

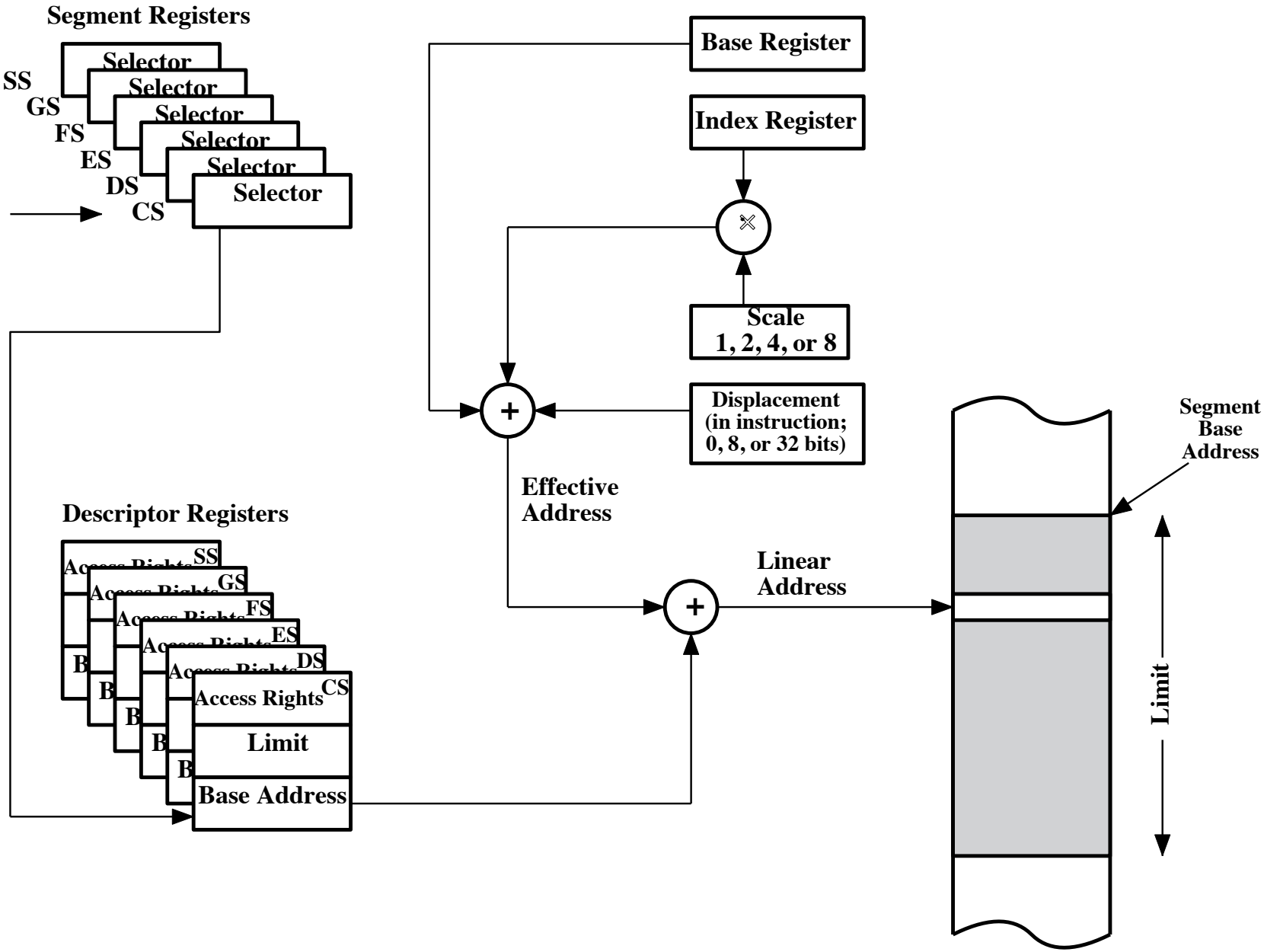


Figure 13.2 x86 Addressing Mode Calculation

x86 Addressing Modes

Mode	Algorithm
Immediate	Operand = A
Register operand	LA = R
Displacement	LA = (SR) + A
Base	LA = (SR) + (B)
Base with displacement	LA = (SR) + (B) + A
Scaled index with displacement	LA = (SR) + (I) · S + A
Base with index and displacement	LA = (SR) + (B) + (I) + A
Base with scaled index and displacement	LA = (SR) + (I) · S + (B) + A
Relative	LA = (PC) + A

- (X): content of register X
- A: content of the address field in the instruction
- S: scaling factor
- SR: segment register
- LA: linear address
- PC: program counter
- R: register
- B: base register
- I: index register

Addressing Modes in Assembly

Using AT&T syntax:

```
displacement(baseReg, offsetReg, scalarMult)
```

which may be more intuitive in Intel syntax:

```
[baseReg + displacement + offsetReg * scalarMult]
```

Examples:

- Load from address $\mathrm{RBP} - 8 + \mathrm{RDX} \cdot 8$ into RAX:

```
movq -8(%rbp, %rdx, 8), %rax
```

- Load the first stack variable into RAX:

```
movq -8(%rbp), %rax
```

- Load the value pointed to by RCX into RDX:

```
movq (%rcx), %rdx
```

More Examples

- Calculate an "address", and store it in RAX:

```
leaq 8(, %rax, 4), %rax
```

- Another "abuse" of the addressing mode to do arithmetic:

```
leaq (%rax, %rax, 2), %rax
```

Notes:

- LEA: load effective address
- Often used as a trick to make faster calculations.
- LEA does not set any flags, the ordinary arithmetic instructions do.
- Generally, an instruction may have at most one address calculation.
  Invalid: `movq (%rax, %rax), (%rbx)`
  Also invalid: `leaq (%rax, %rax), (%rbx)`

- Introduction
- Where are we going with this?
- x86-64 Data Types and Register Organisation
- Instructions Basics
- Programs, Libraries, and the Operating System
- Putting It Together
- Addressing Memory
- Stack and Function Calls

Function Operations

- Calling
- Returning
- Returning a value
- Passing arguments
- Storing local variables when in a function
- Handle registers, well-defined interference

First Attempt

```
funcOne:
    ...
    jmp funcTwo
returnPointFuncOne:
    ...

funcTwo:
    ...
    jmp returnPointFuncOne
```

- It only works in this specific case.
- But it's not general.
- We may want to call the same function from different places.
- There is no way to distinguish between different callers.

Second Attempt

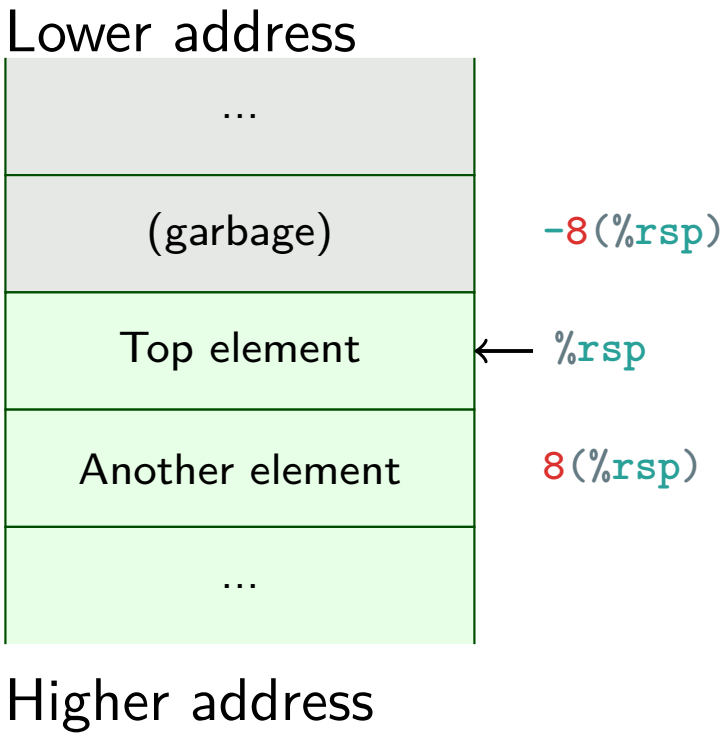
```
funcA:
    ...
    movq $returnPointA, %rax
    jmp funcF
returnPointA:
    ...

funcB:
    ...
    movq $returnPointB, %rax
    jmp funcF
returnPointB:
    ...

funcF:
    ...
    jmp *%rax
```

- `jmp *%`**reg** means "jump to the address stored in REG" (an indirect jump/branch)
- Now we can return to the right place.
- But what if `funcF` wants to call another function?
- And what about recursive functions?

Solution: Use the Stack

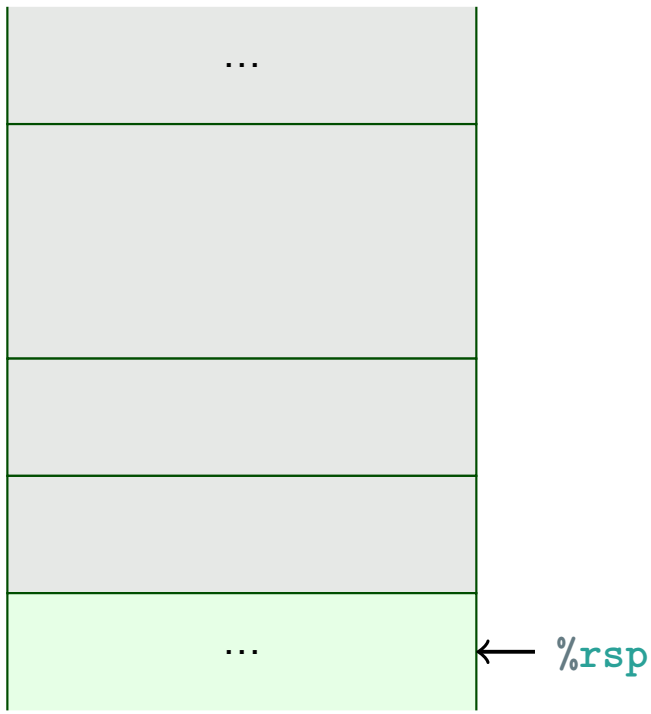


- The stack grows from high to low addresses.
- Where RSP points is the top of the stack.
- `push` **operand**:
  1. Decrease `%rsp`.
  2. Store **operand** in `(%rsp)`.
- `pop` **operand**:
  1. Fetch `(%rsp)` into **operand**.
  2. Increase `%rsp`.

Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

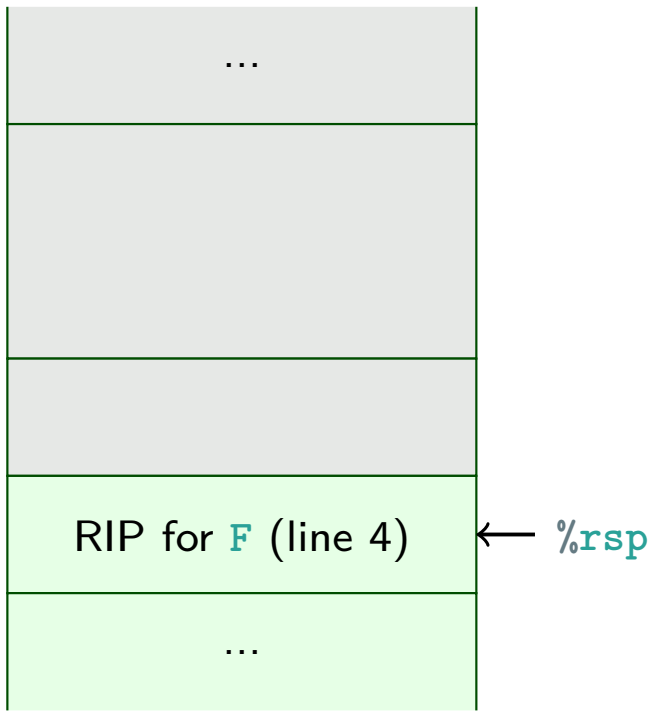
```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

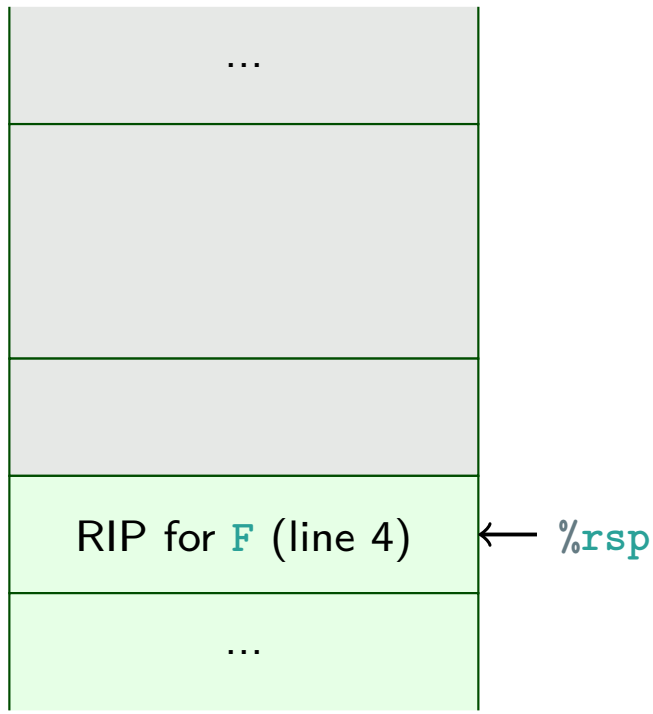
```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

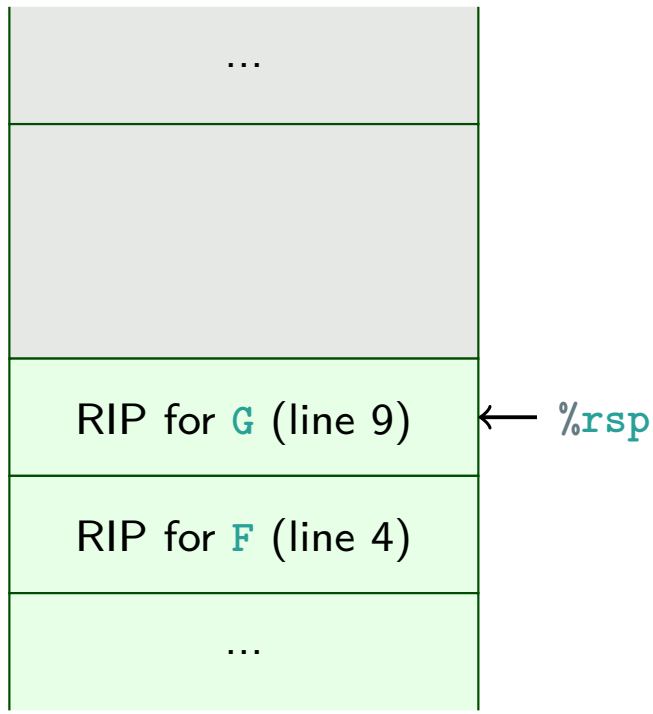
```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

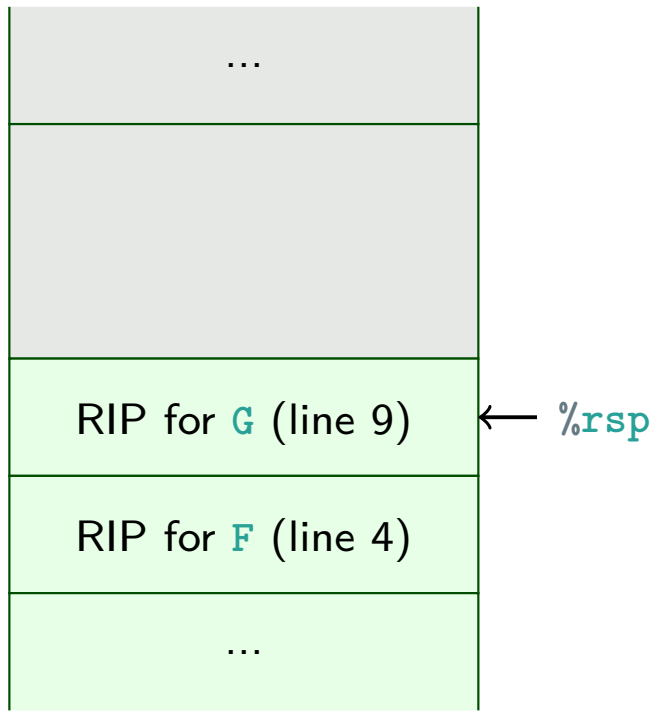
```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

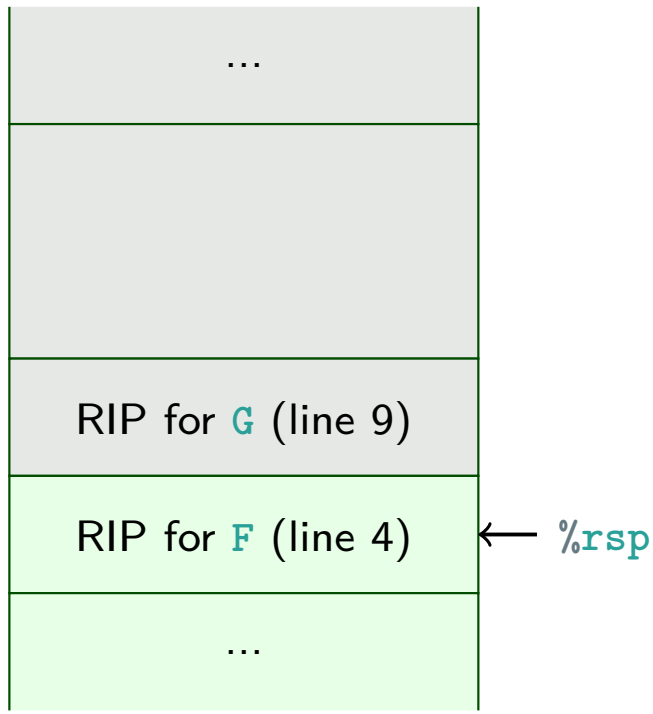
```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

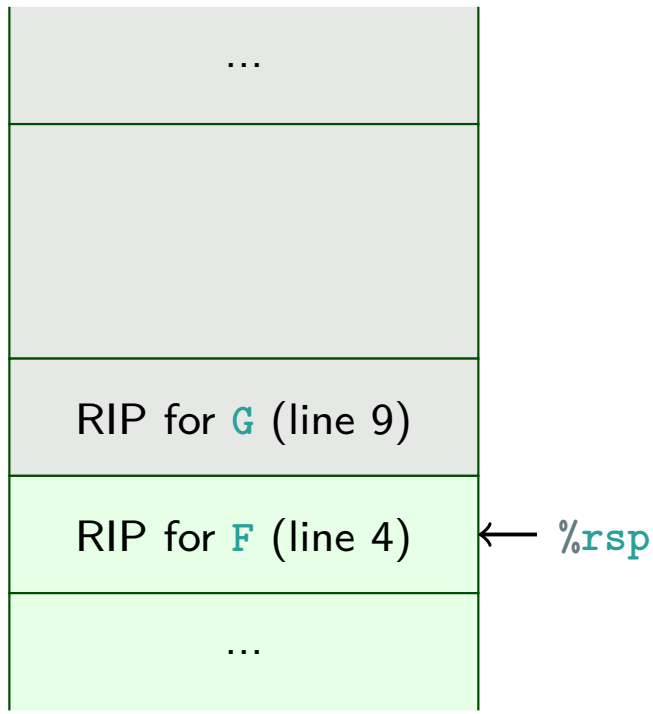
```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

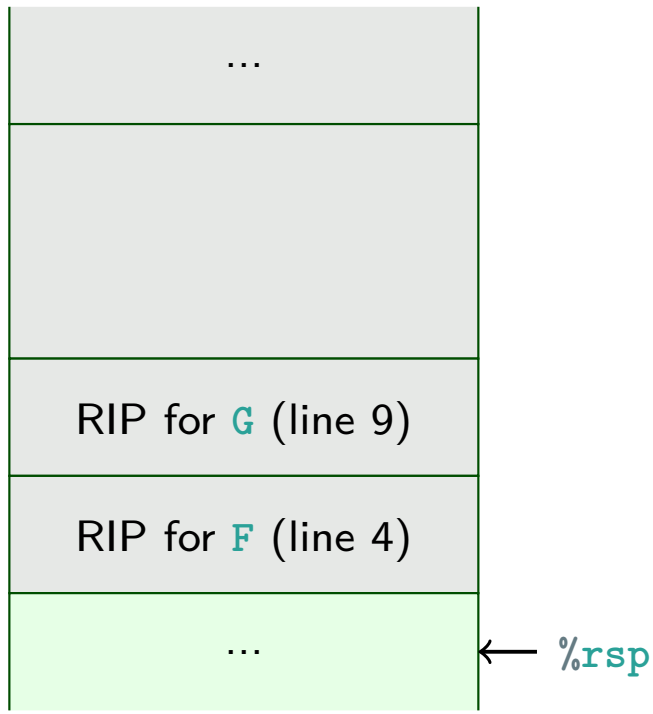
```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

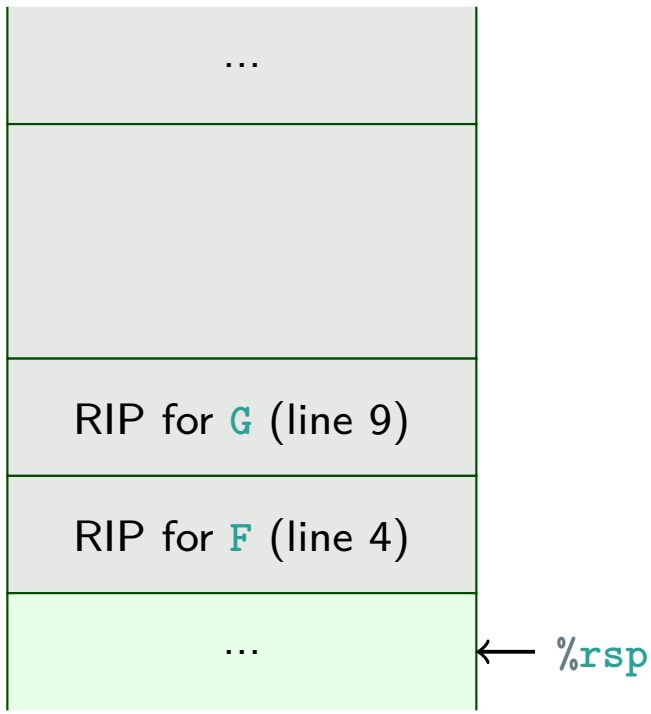
```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



Calling and Returning

- `call` **label**:
  1. Push the instruction pointer, RIP.
  2. Jump to **label**
- `ret`:
  1. Pop into RIP.

```
1   F:
2       ...
3       call G
4       ...
5
6   G:
7       ...
8       call H
9       ...
10      ret
11
12  H:
13      ...
14      ret
```



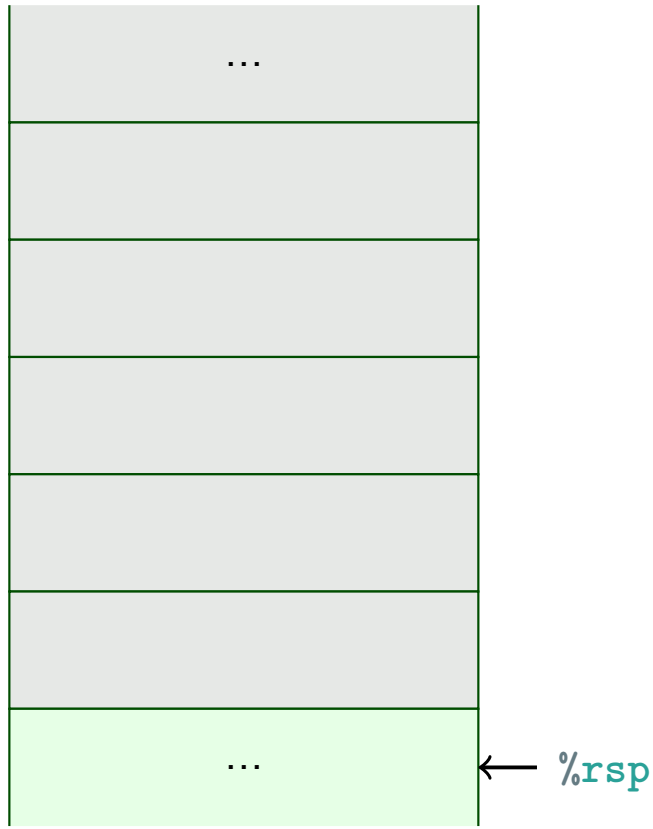
Passing Arguments and Returning Data

- There are multiple calling conventions.
- In x86-64 Linux programs (System V AMD64 ABI):
  - The first 6 integer/pointer arguments in registers.
  - If more, on the stack.
  - Order: RDI, RSI, RDX, RCX, R8, R9
- In x86 Linux programs (cdecl convention):
  - Put all arguments on the stack.
  - In reverse order.
- Return value (if an integer/pointer): in RAX.

Passing Arguments: cdecl convention

```
1   F:
2       ...
3       push argN
4       ...
5       push arg1
6       call G
7       addq $xn, %rsp
8       ...
9
10  G:
11      ...
12      ret
```

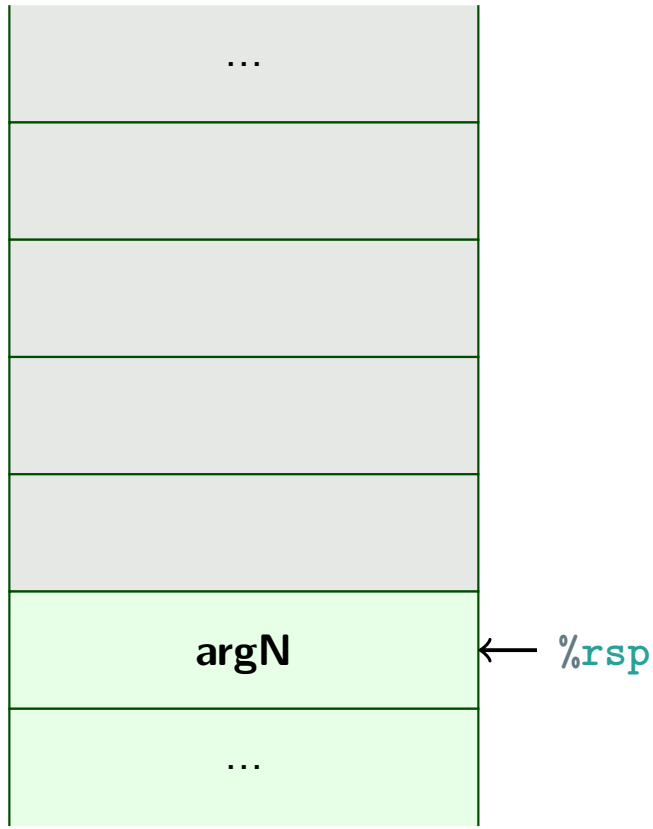
With **xn** $= \mathrm{N} \cdot 8$
(for 64-bit programs).



Passing Arguments: cdecl convention

```
1   F:
2       ...
3       push argN
4       ...
5       push arg1
6       call G
7       addq $xn, %rsp
8       ...
9
10  G:
11      ...
12      ret
```

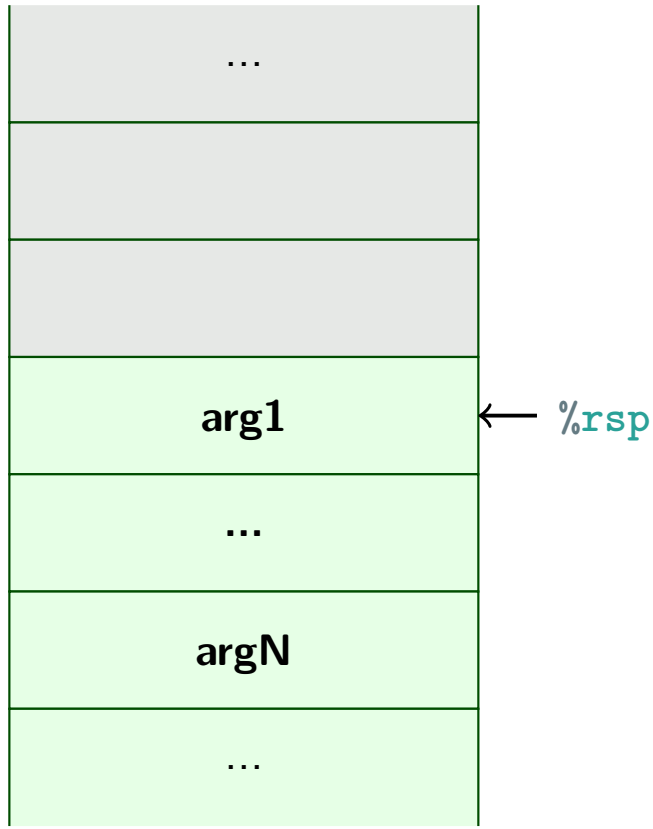
With **xn** $= \mathrm{N} \cdot 8$
(for 64-bit programs).



Passing Arguments: cdecl convention

```
1   F:
2       ...
3       push argN
4       ...
5       push arg1
6       call G
7       addq $xn, %rsp
8       ...
9
10  G:
11      ...
12      ret
```

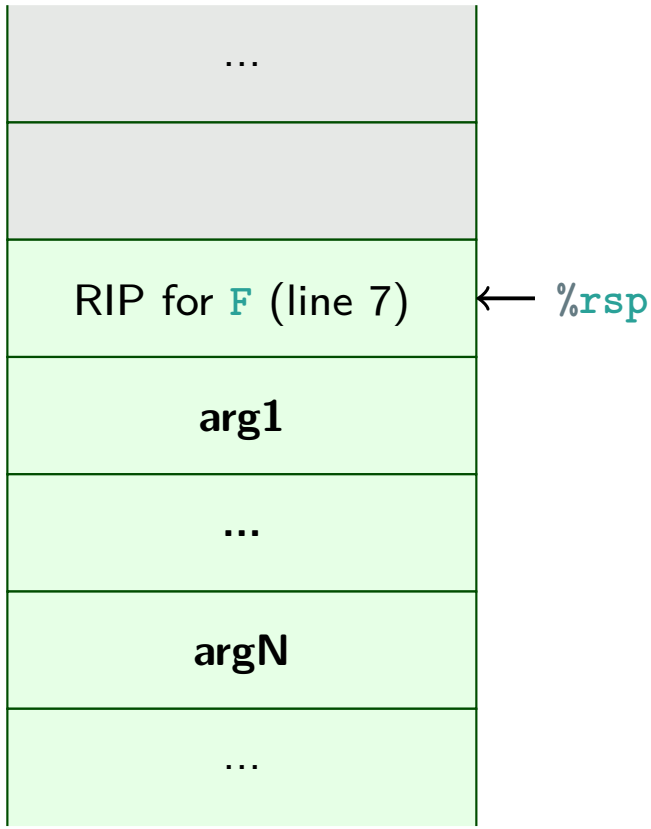
With **xn** $= \mathrm{N} \cdot 8$
(for 64-bit programs).



Passing Arguments: cdecl convention

```
1   F:
2       ...
3       push argN
4       ...
5       push arg1
6       call G
7       addq $xn, %rsp
8       ...
9
10  G:
11      ...
12      ret
```

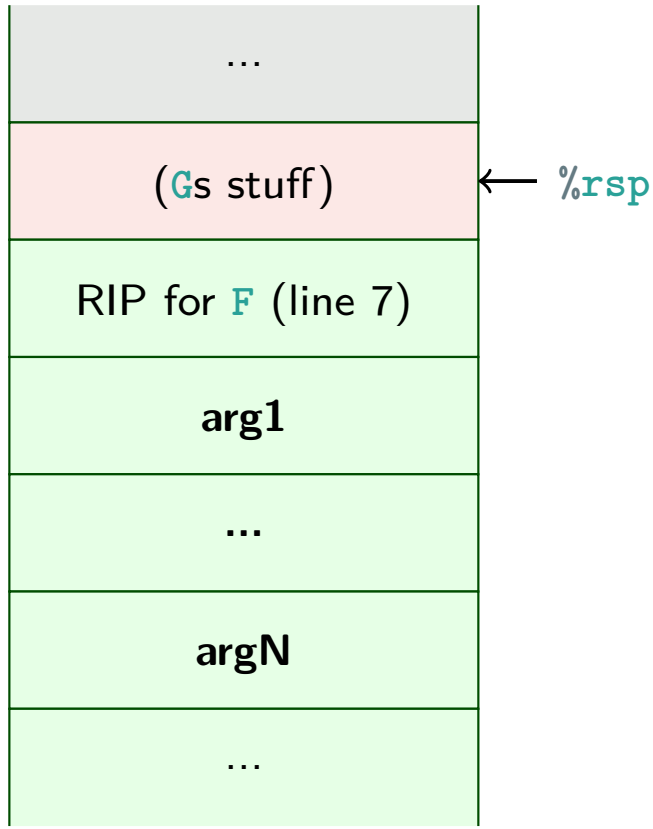
With **xn** $= \mathrm{N} \cdot 8$
(for 64-bit programs).



Passing Arguments: cdecl convention

```
1   F:
2       ...
3       push argN
4       ...
5       push arg1
6       call G
7       addq $xn, %rsp
8       ...
9
10  G:
11      ...
12      ret
```

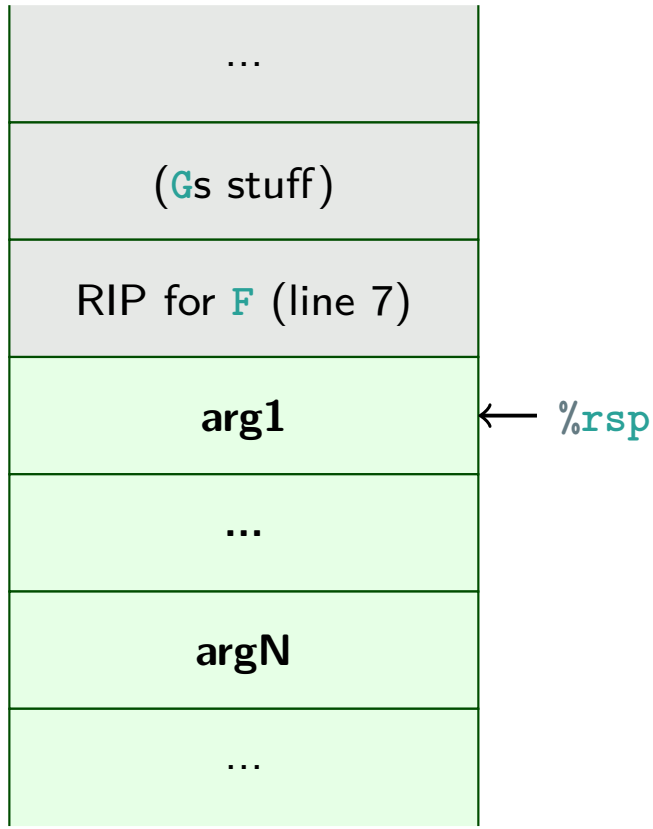
With **xn** $= \mathrm{N} \cdot 8$
(for 64-bit programs).



Passing Arguments: cdecl convention

```
1   F:
2       ...
3       push argN
4       ...
5       push arg1
6       call G
7       addq $xn, %rsp
8       ...
9
10  G:
11      ...
12      ret
```

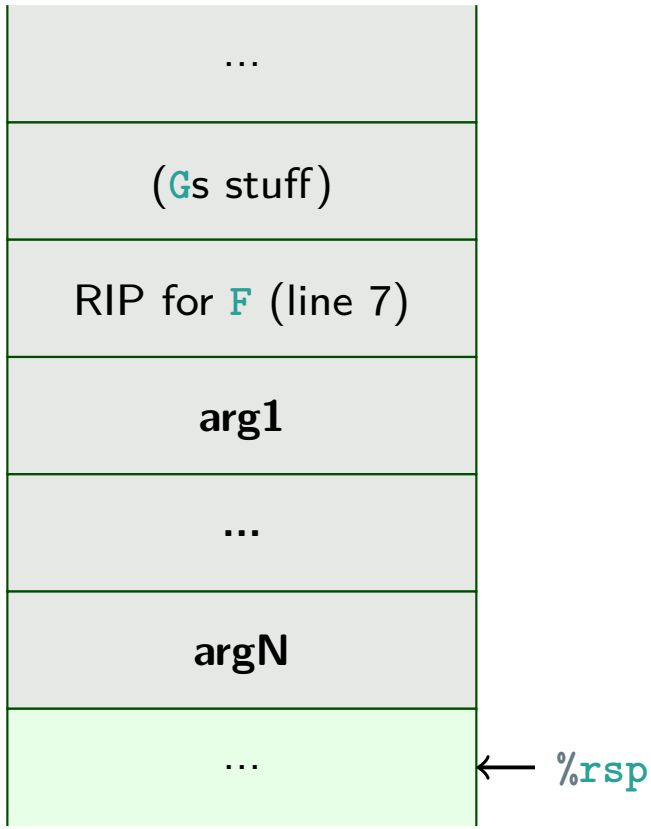
With **xn** $= \mathrm{N} \cdot 8$
(for 64-bit programs).



Passing Arguments: cdecl convention

```
1   F:
2       ...
3       push argN
4       ...
5       push arg1
6       call G
7       addq $xn, %rsp
8       ...
9
10  G:
11      ...
12      ret
```

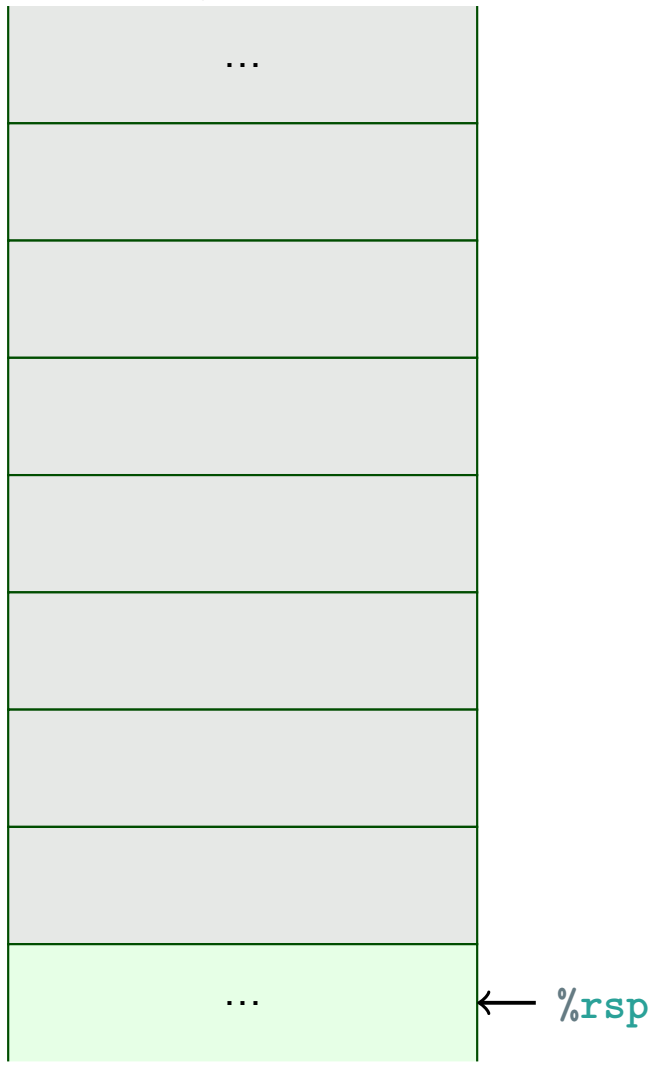
With **xn** $= \mathrm{N} \cdot 8$
(for 64-bit programs).



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

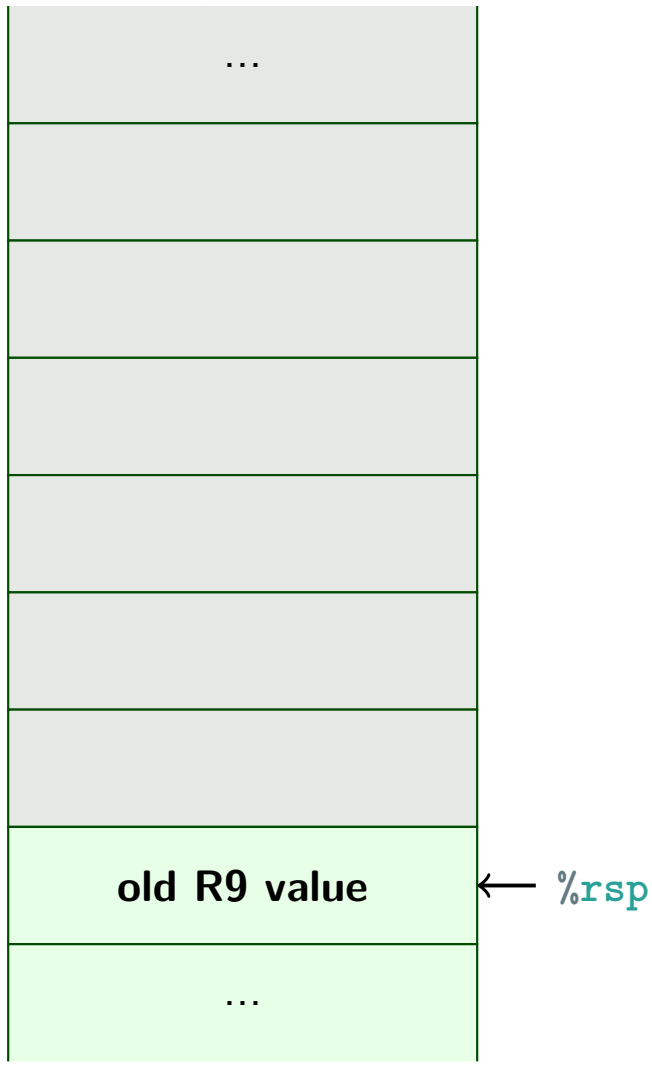
This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

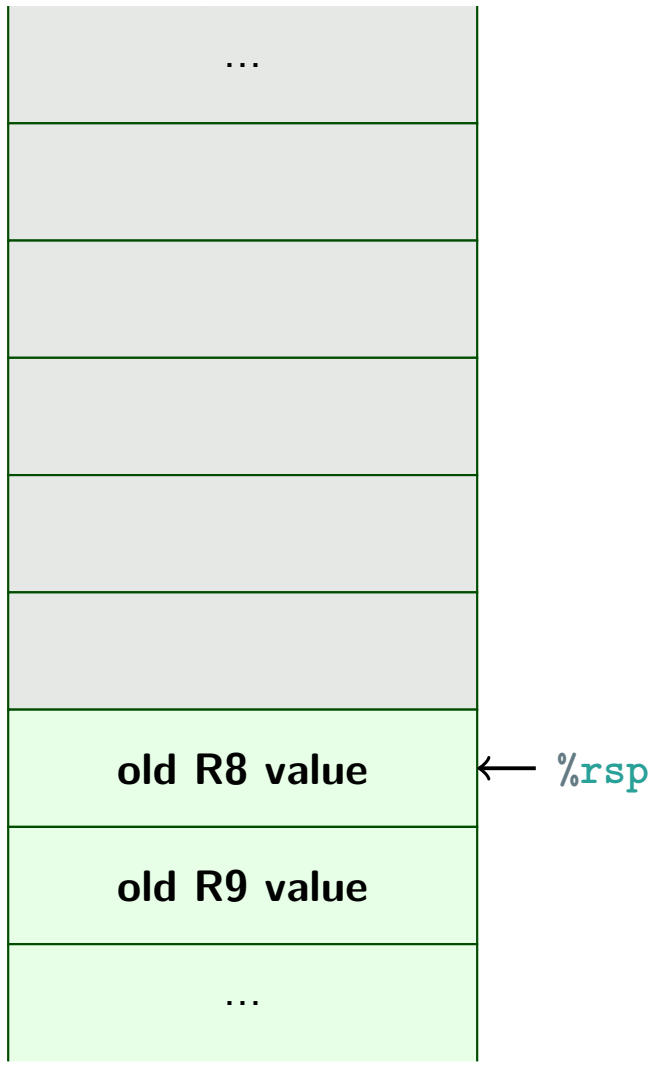
This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

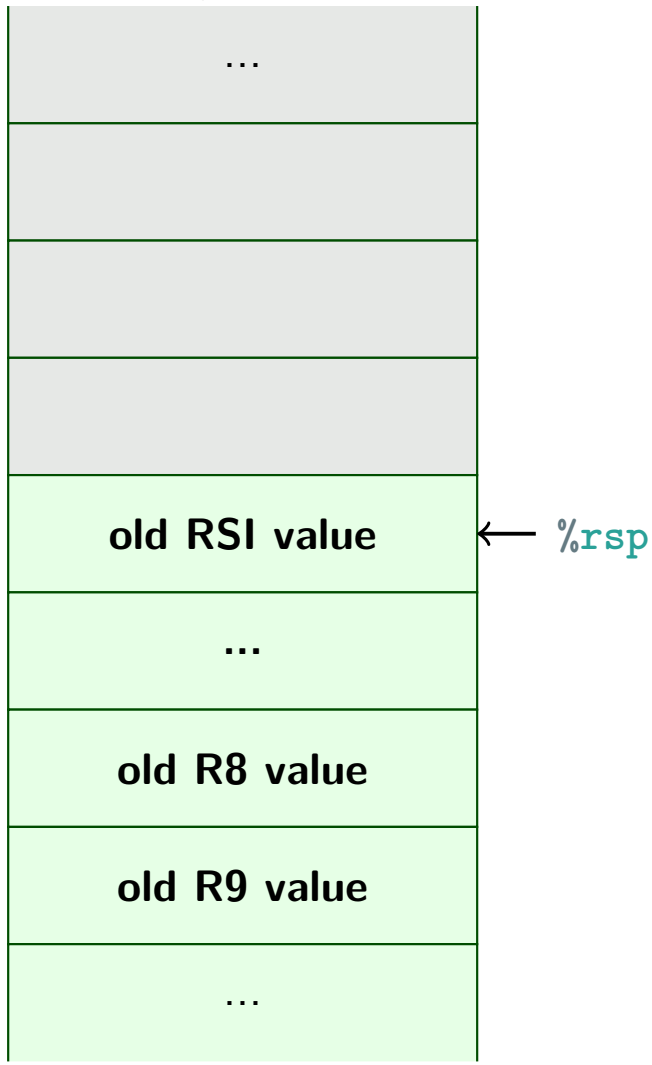
This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

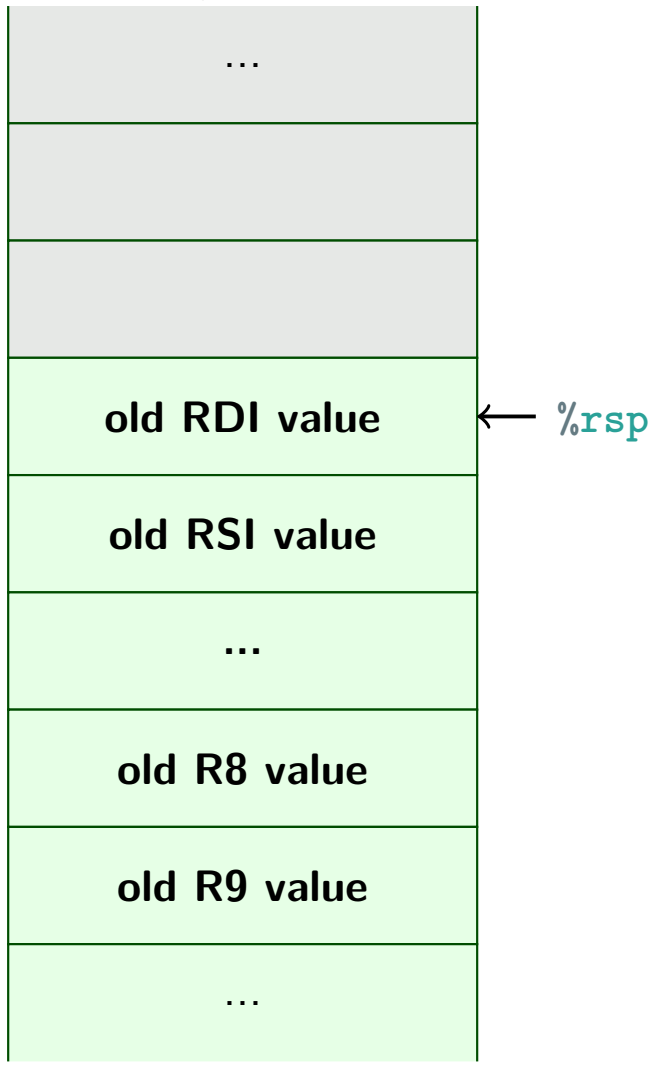
This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

This is the convention you will use.

Passing Arguments, example: System V convention

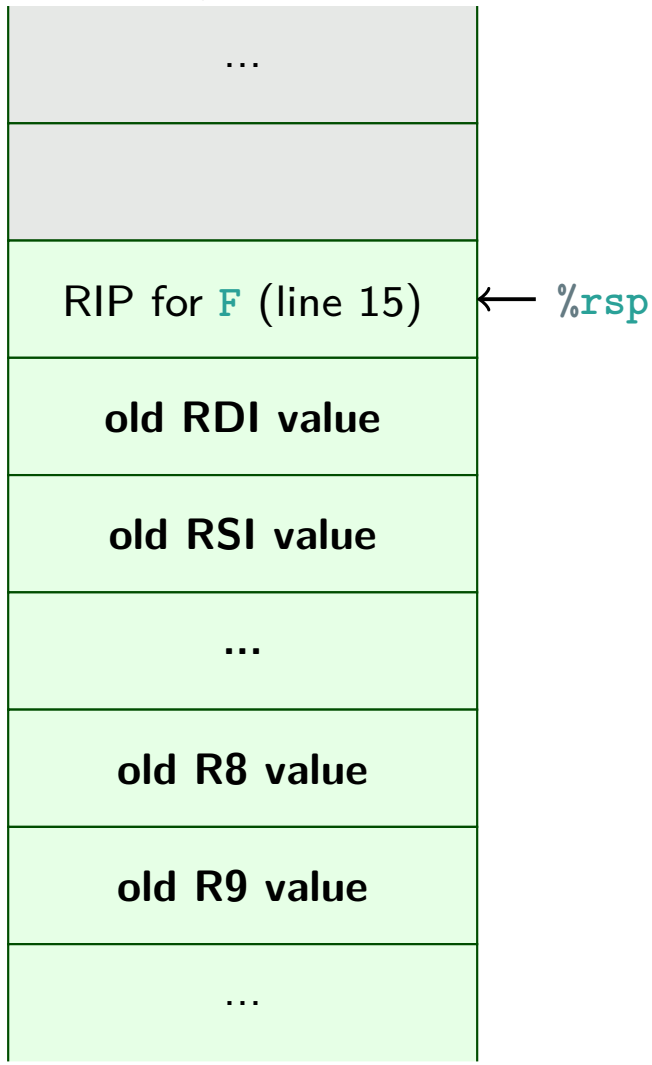
```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

This is the convention you will use.

Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

This is the convention you will use.



Passing Arguments, example: System V convention

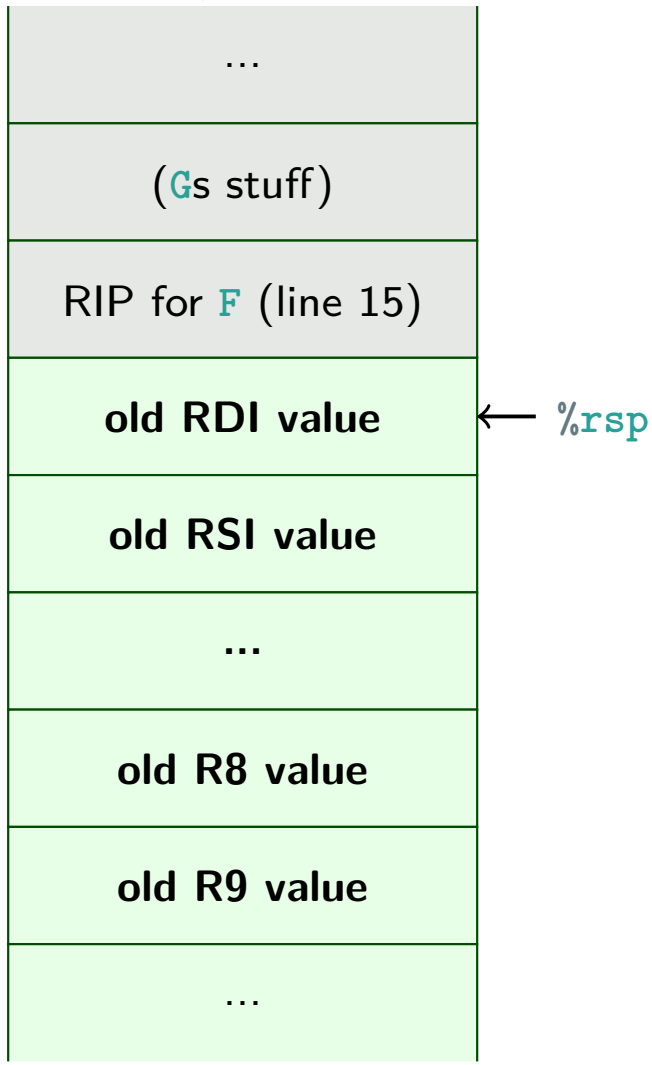
```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

This is the convention you will use.

Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

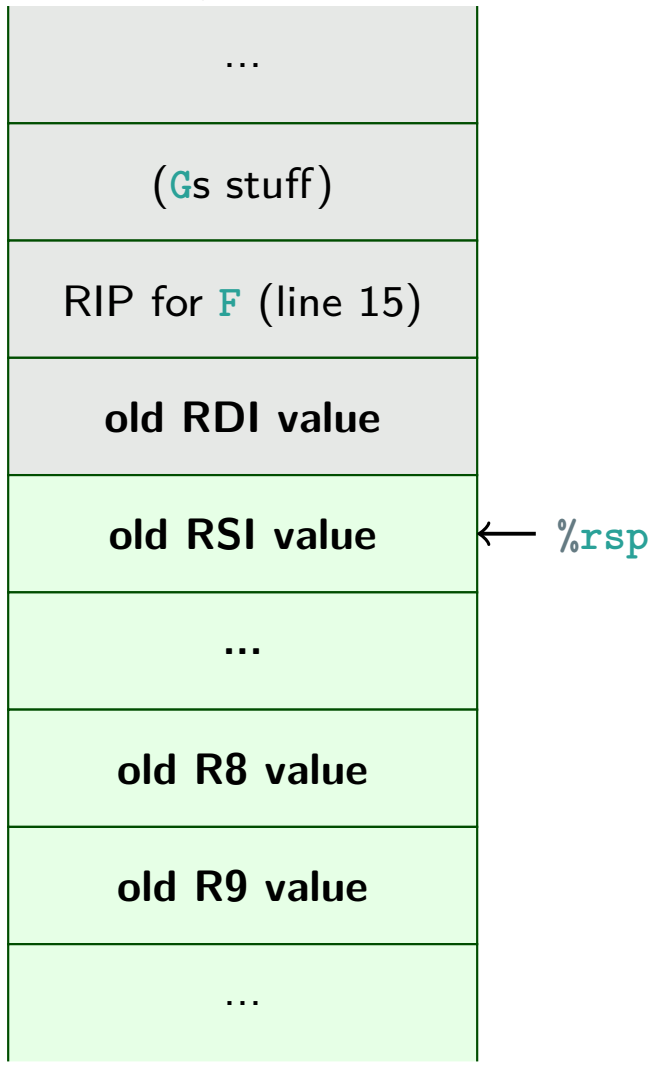
This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

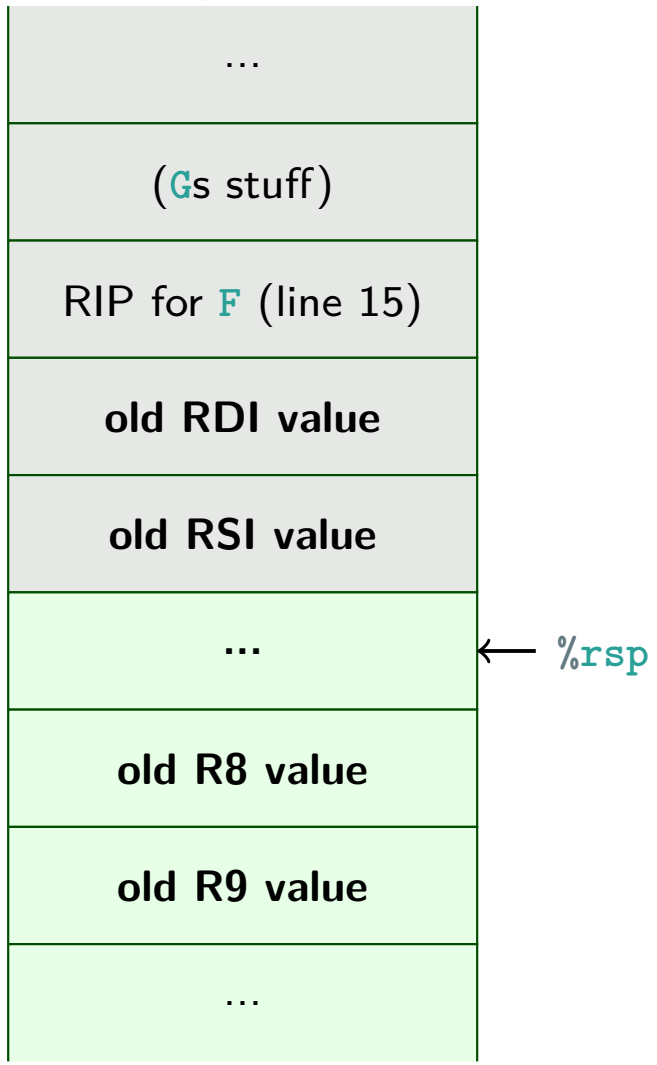
This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

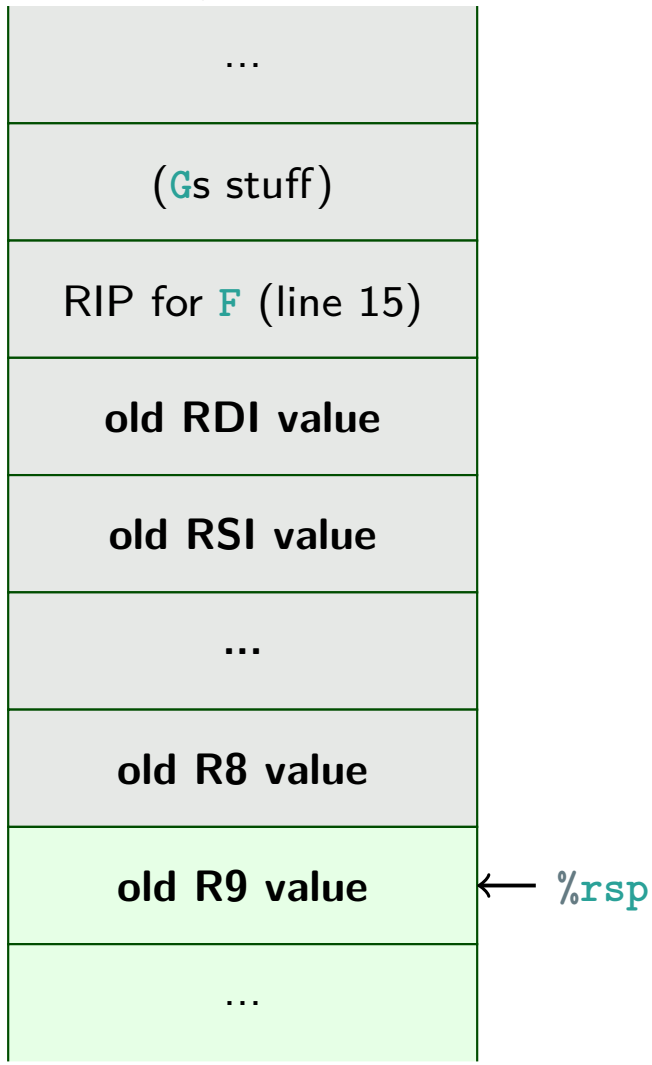
This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

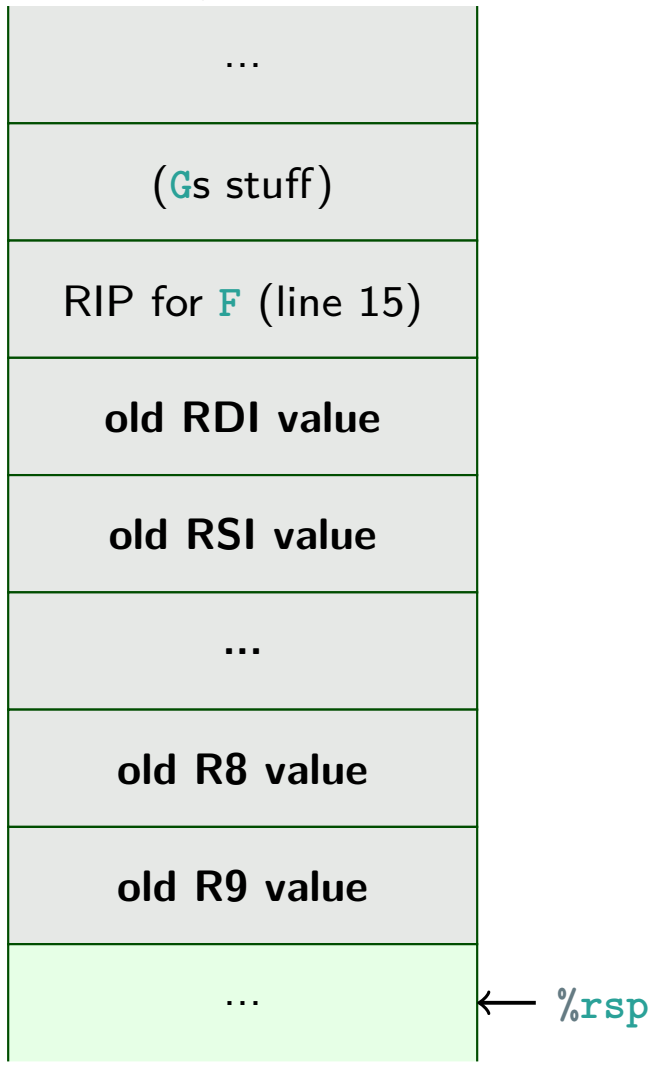
This is the convention you will use.



Passing Arguments, example: System V convention

```
1   F:
2       ...
3   # save old register values
4       push %r9
5       push %r8
6       ...
7       push %rsi
8       push %rdi
9   # move args into regiters
10      movq <something>, %r9
11      ...
12      movq <something>, %rdi
13      call G
14  # restore old register values
15      pop %rdi
16      pop %rsi
17      ...
18      pop %r8
19      pop %r9
20      ...
21
22  G:
23      ...
24      ret
```

This is the convention you will use.

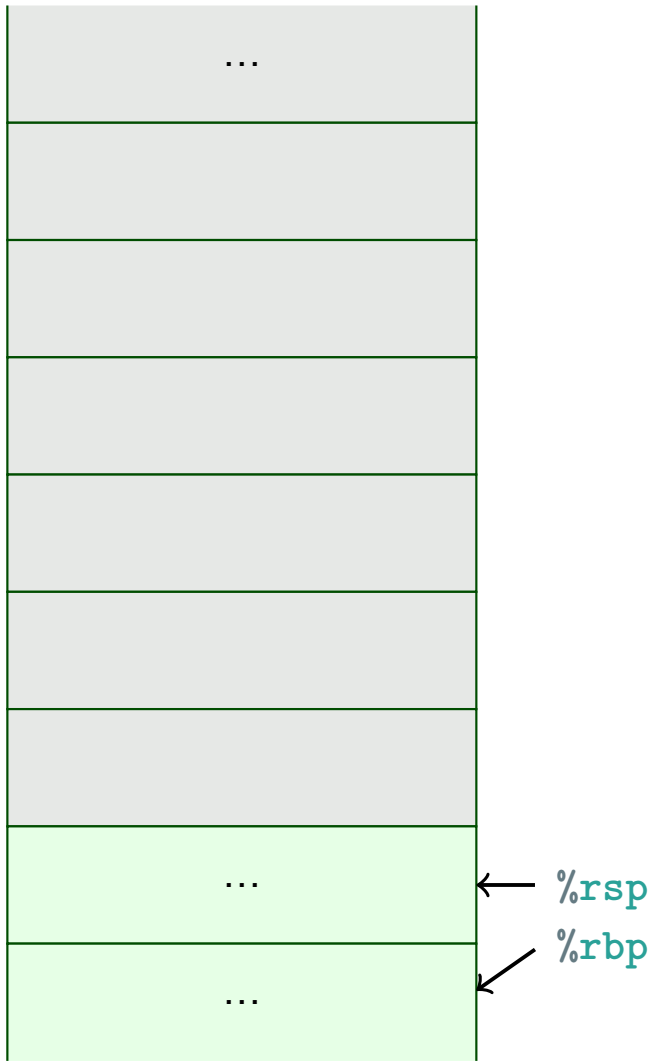


Local Variables and The Stack Base Pointer

- In a function we would like to have local variables.
- We could put them on the stack (if we run out of registers).
- But the stack pointer may be changing all the time, so where are the variables?
- Let's keep a pointer to where our part of the stack started: register RBP.
- First task in a function: save the old RBP, and establish a new one.
- Last task in a function: restore RSP and RBP.
- Arguments and local variables are now constant offsets from RBP!

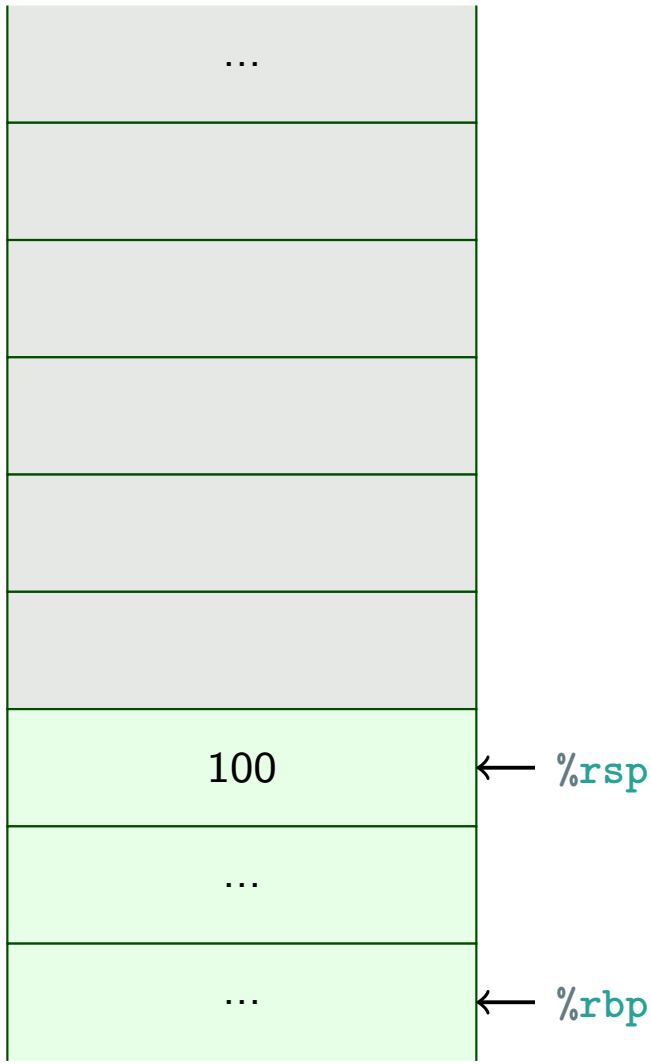
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



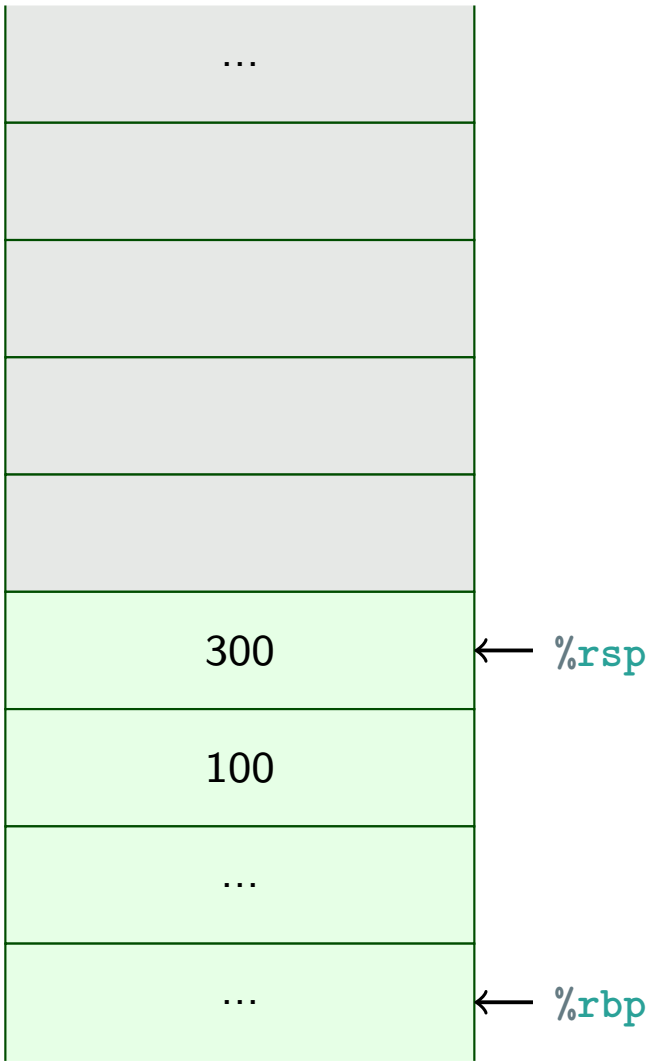
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



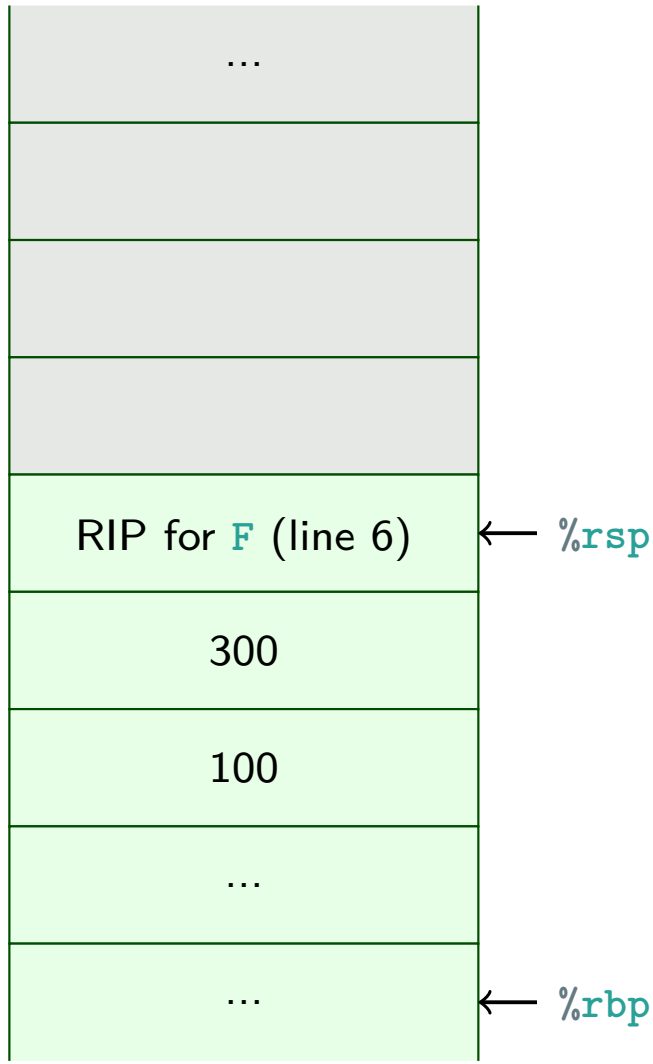
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



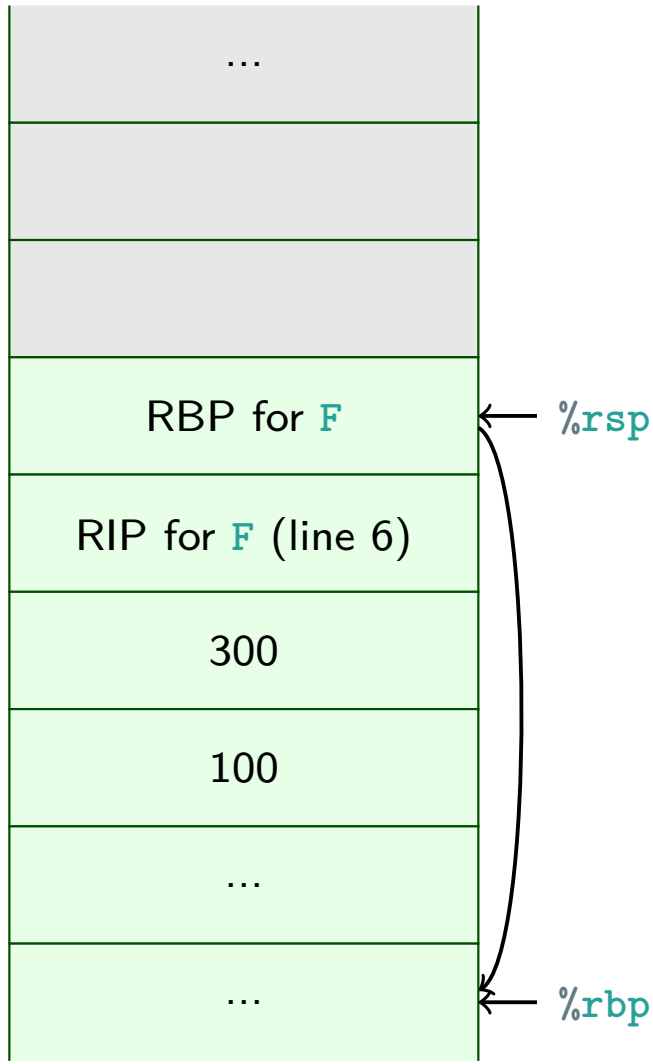
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



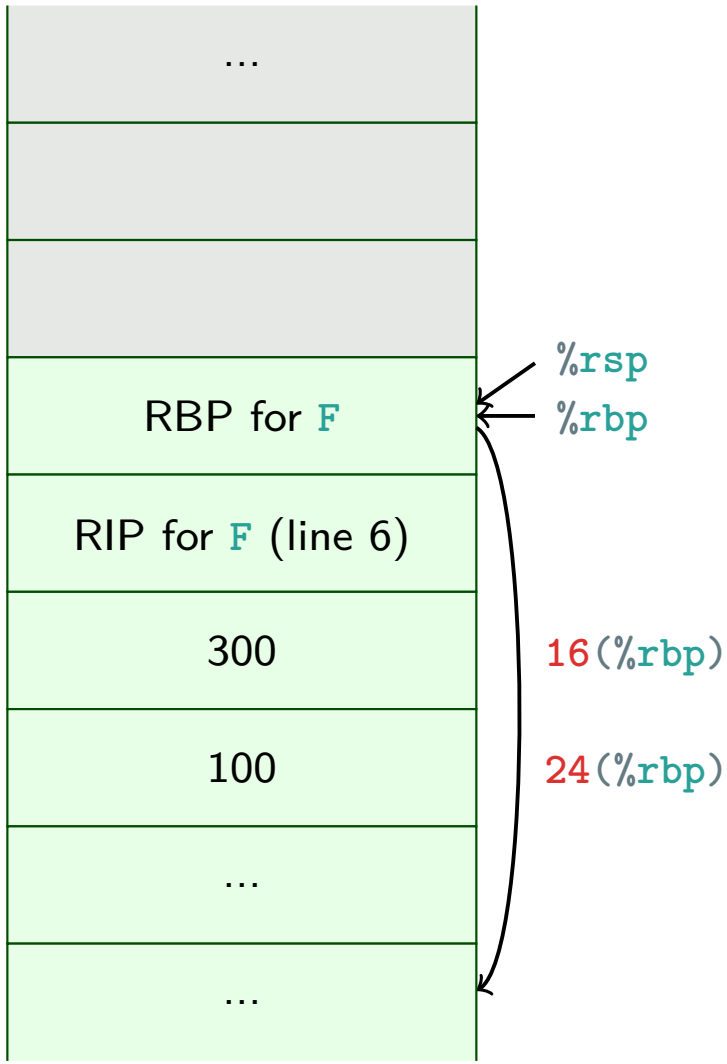
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



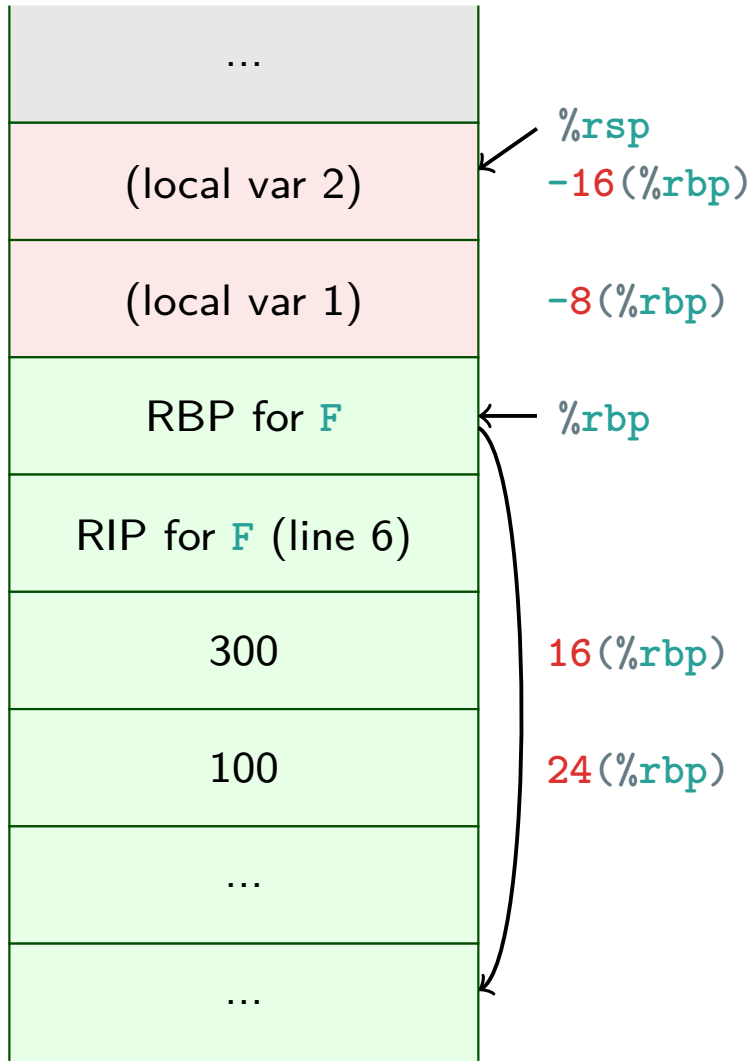
Example of Stack Frames, cdecl convention

```
1  F:
2      ...
3      push $100
4      push $300
5      call G
6      addq $16, %rsp
7      ...
8
9  G:
10     push %rbp
11     movq %rsp, %rbp
12     subq $16, %rsp
13
14     movq 16(%rbp), %rax
15     addq 24(%rbp), %rax
16
17     movq %rbp, %rsp
18     pop %rbp
19     ret
```



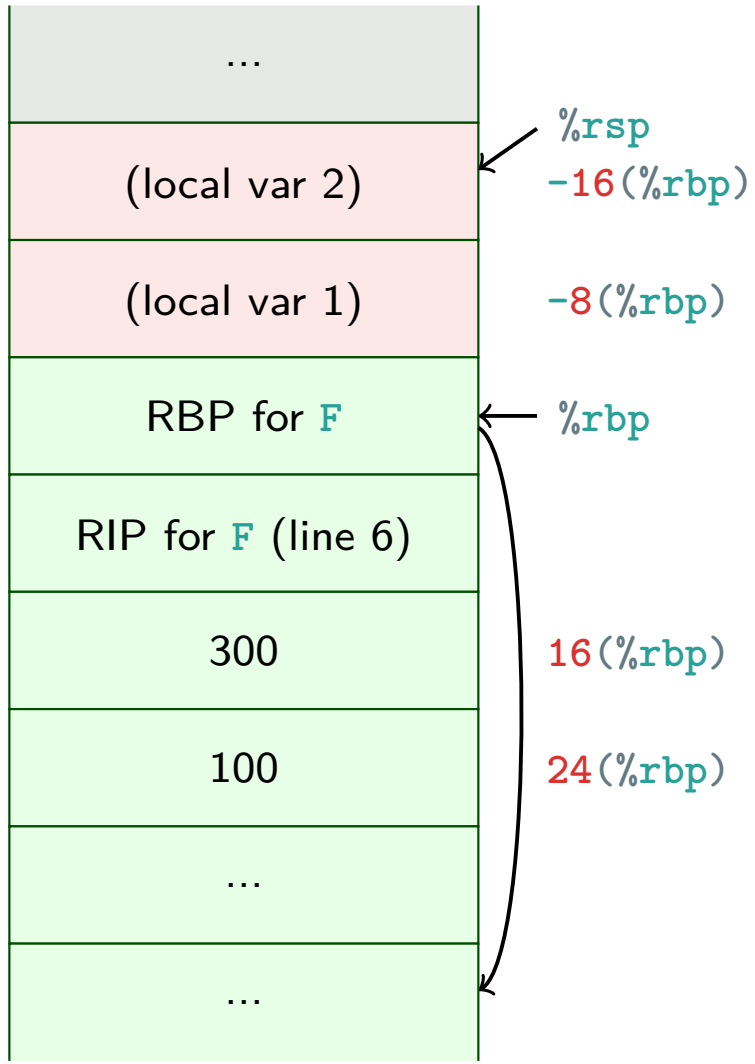
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



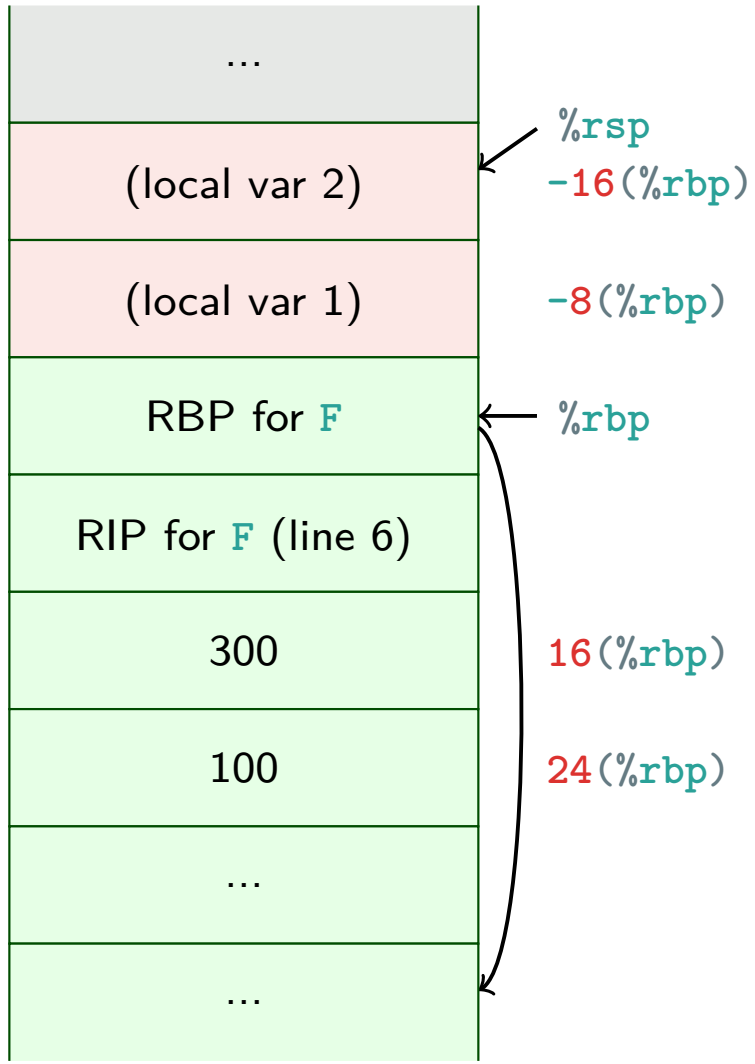
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



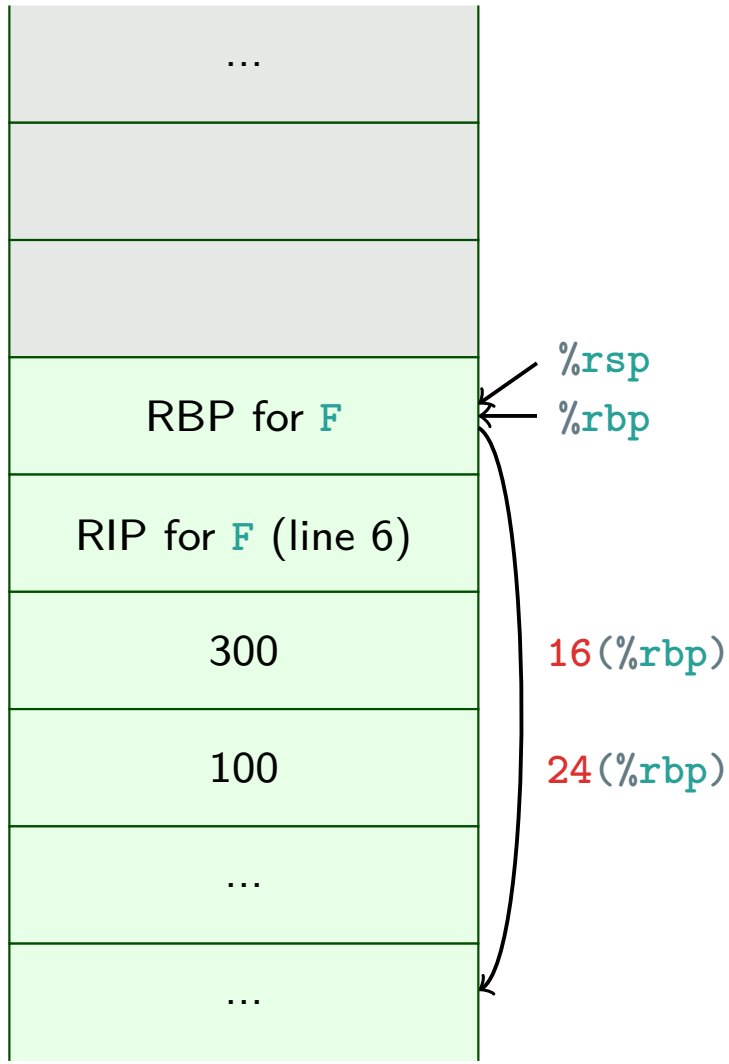
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



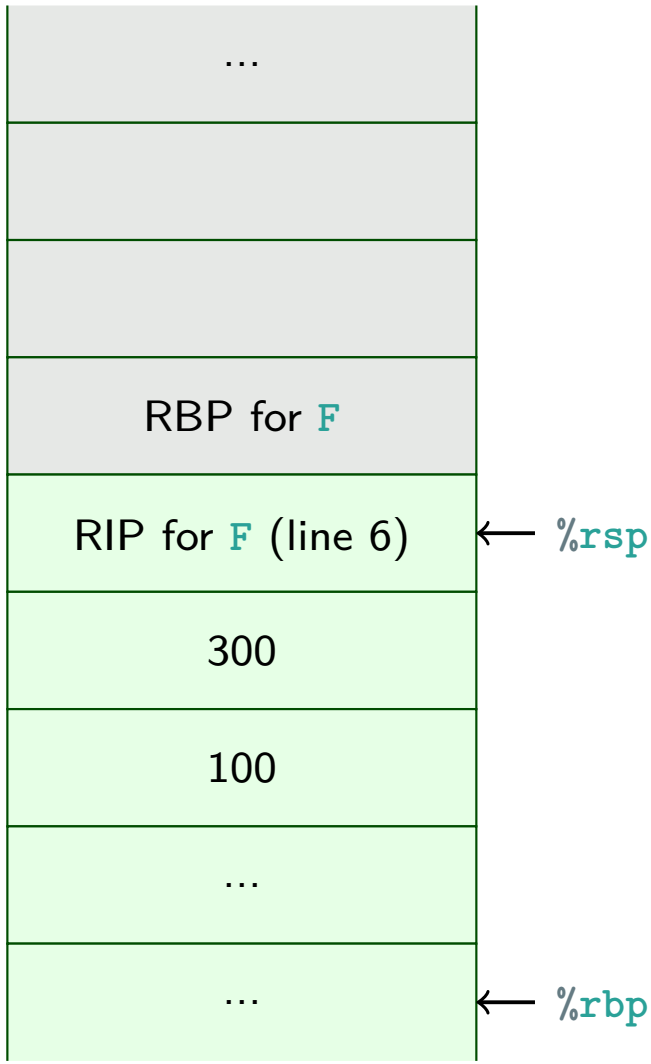
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



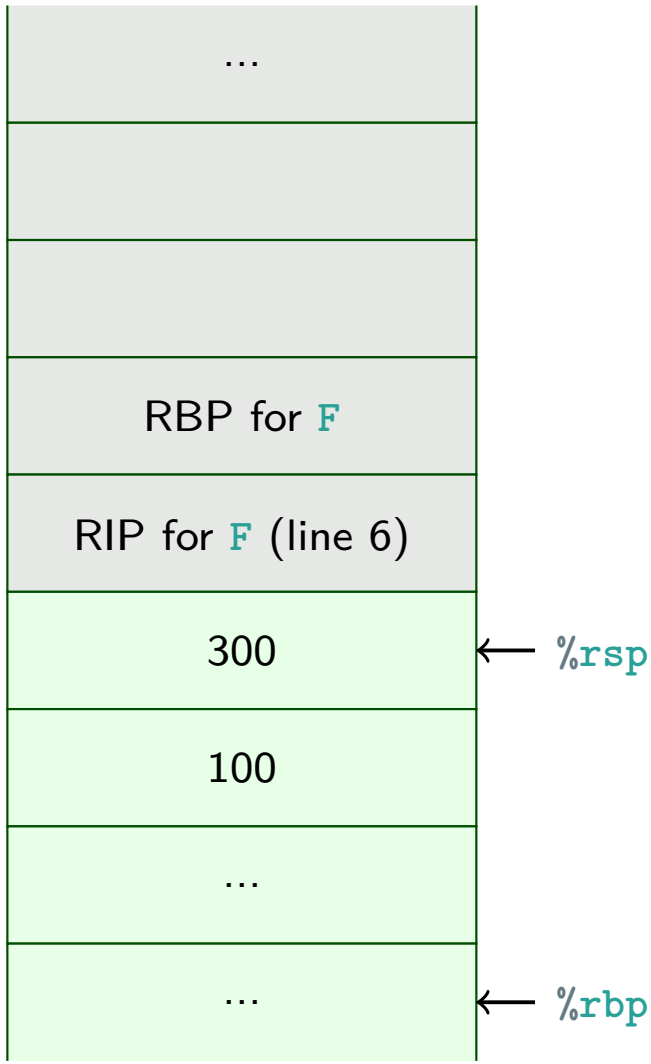
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



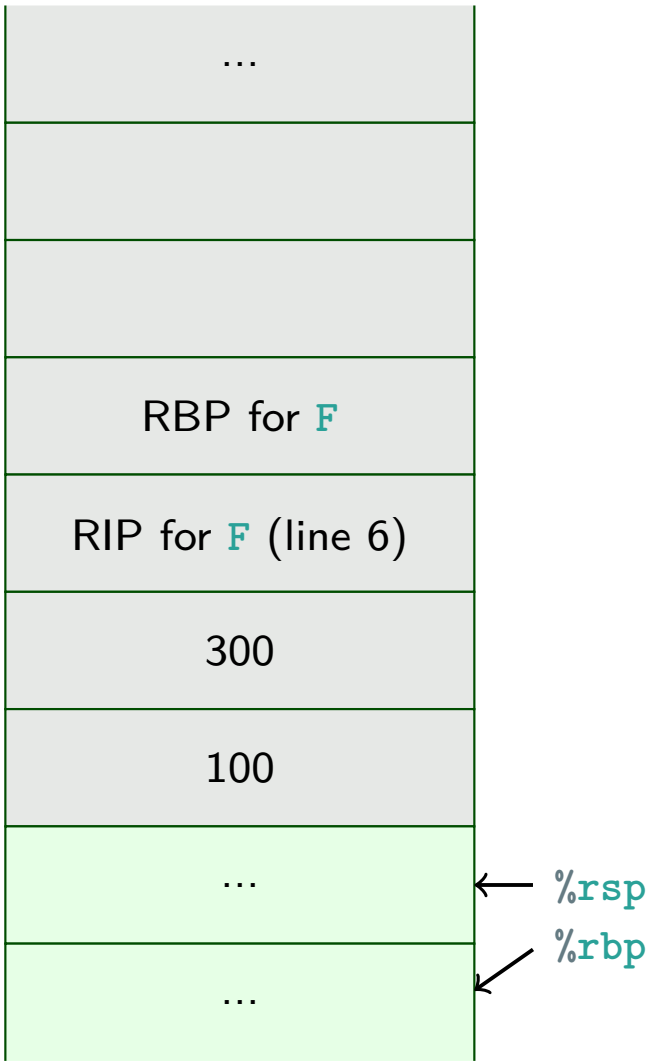
Example of Stack Frames, cdecl convention

```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19  →   ret
```



Example of Stack Frames, cdecl convention

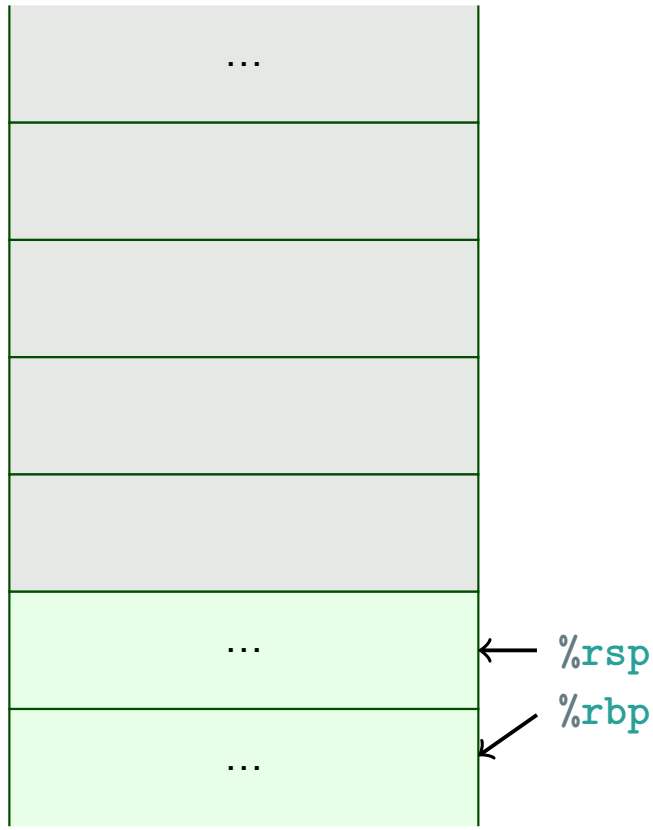
```
1   F:
2       ...
3       push $100
4       push $300
5       call G
6       addq $16, %rsp
7       ...
8
9   G:
10      push %rbp
11      movq %rsp, %rbp
12      subq $16, %rsp
13
14      movq 16(%rbp), %rax
15      addq 24(%rbp), %rax
16
17      movq %rbp, %rsp
18      pop %rbp
19      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

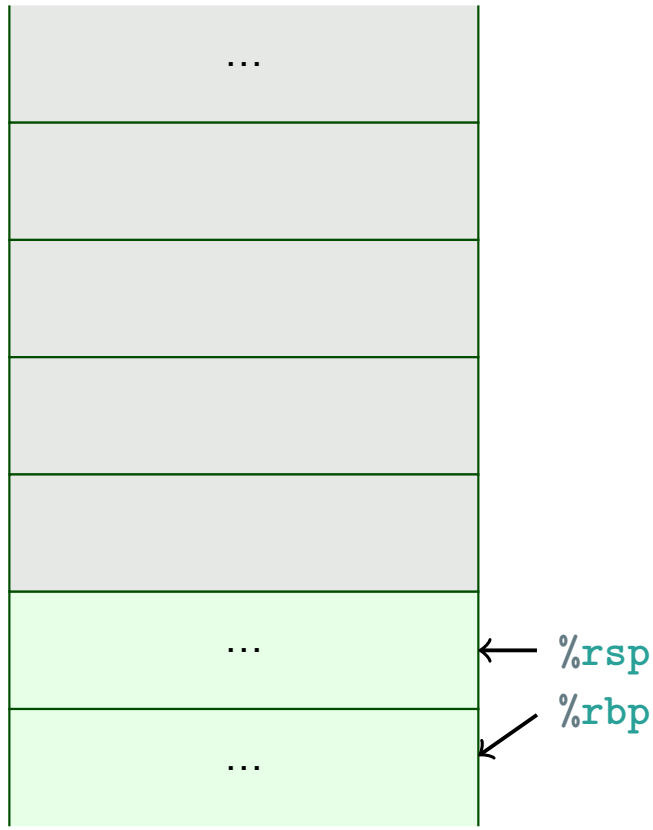
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

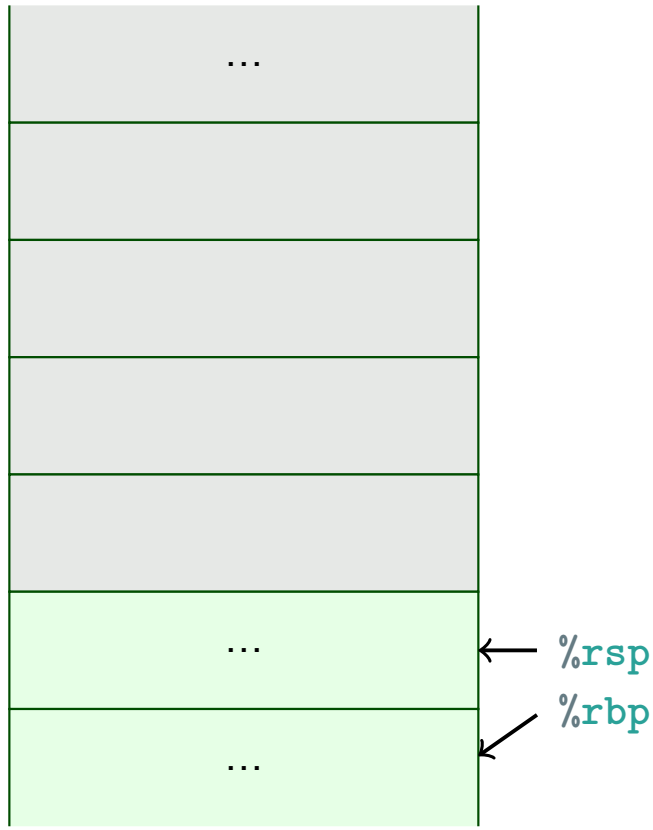
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

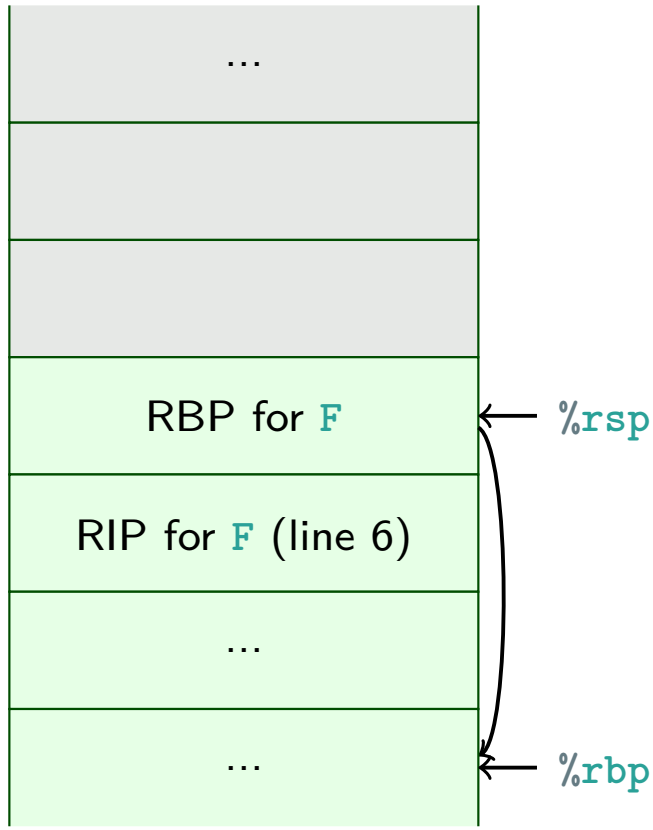
This is the convention you will use.

```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```

Example of Stack Frames, System V convention

This is the convention you will use.

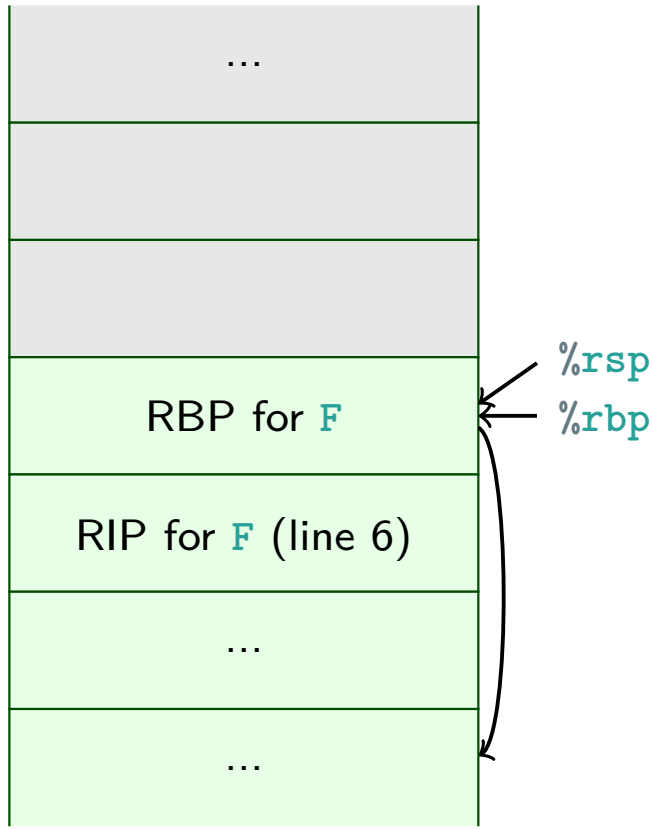
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

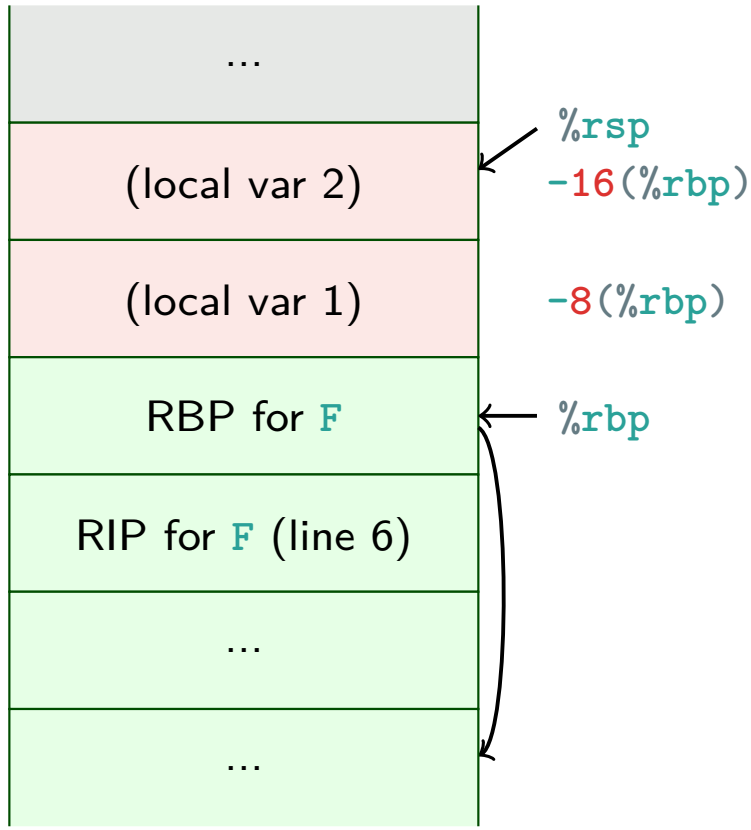
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

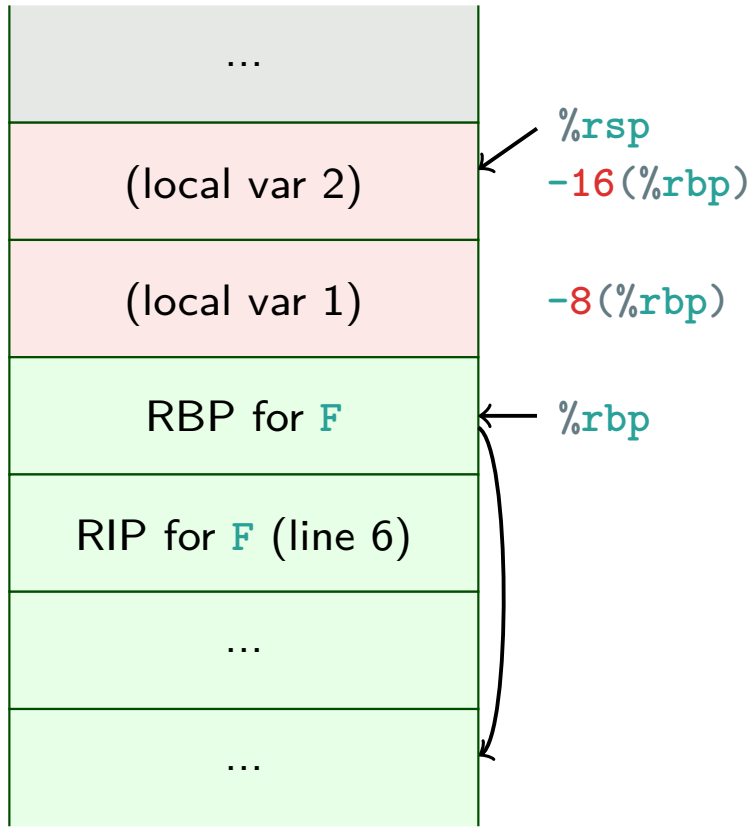
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

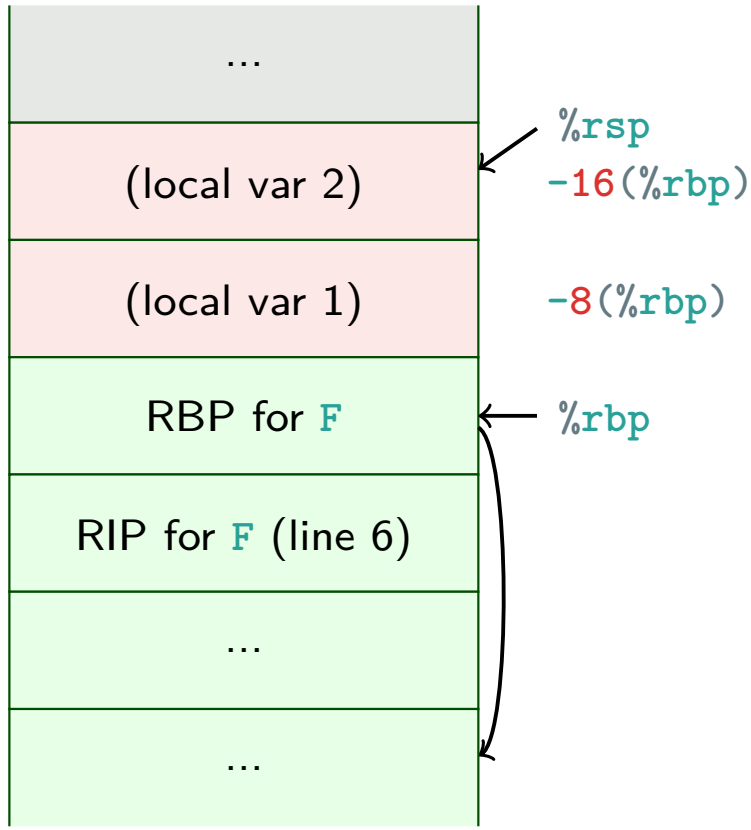
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

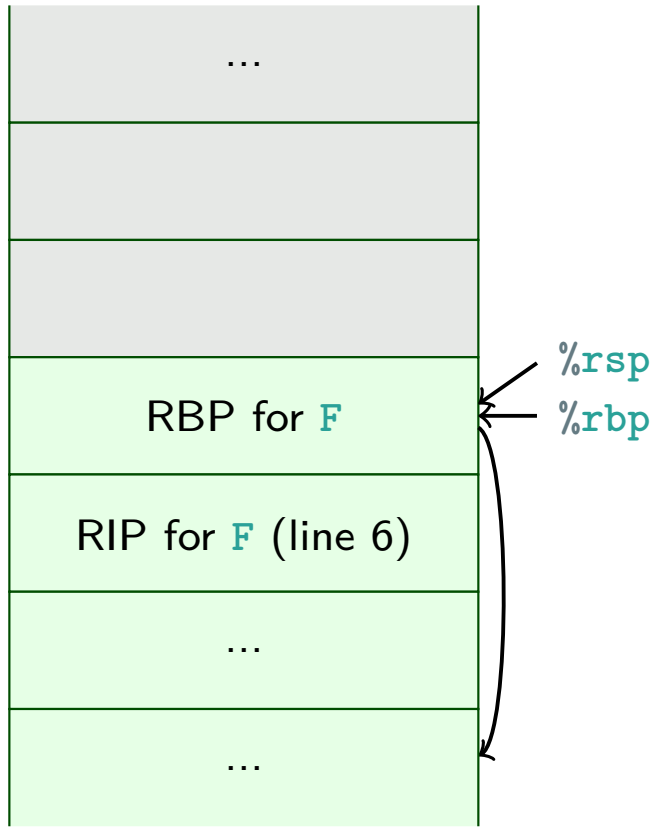
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

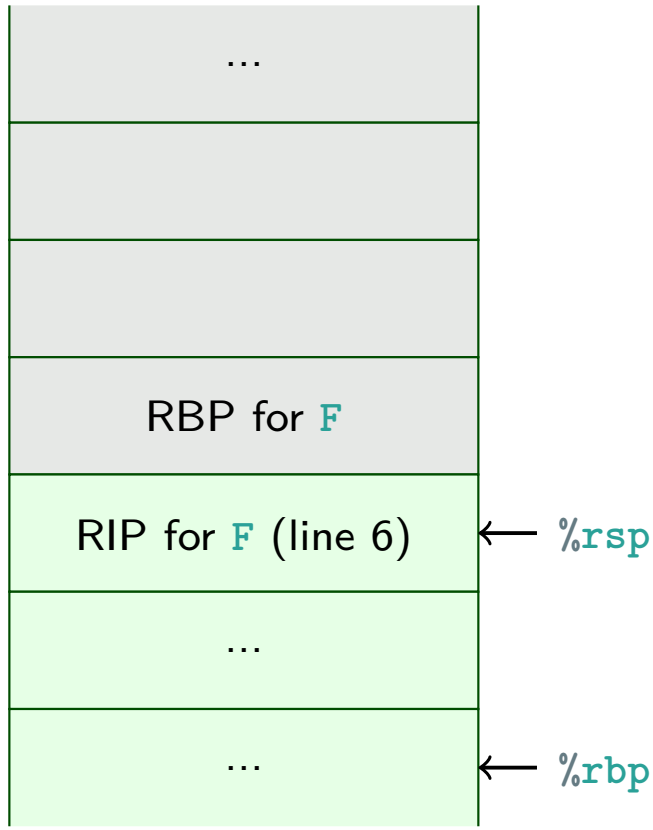
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

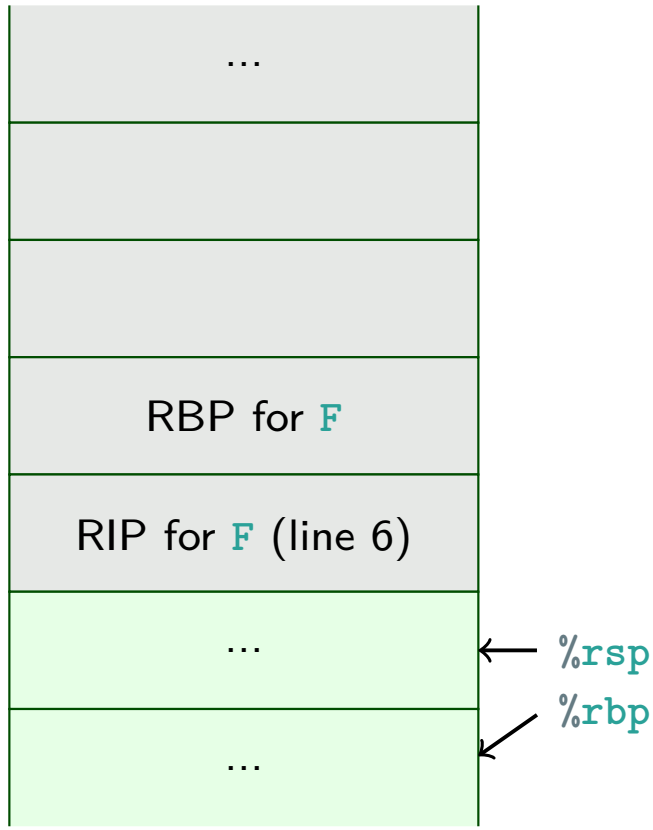
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

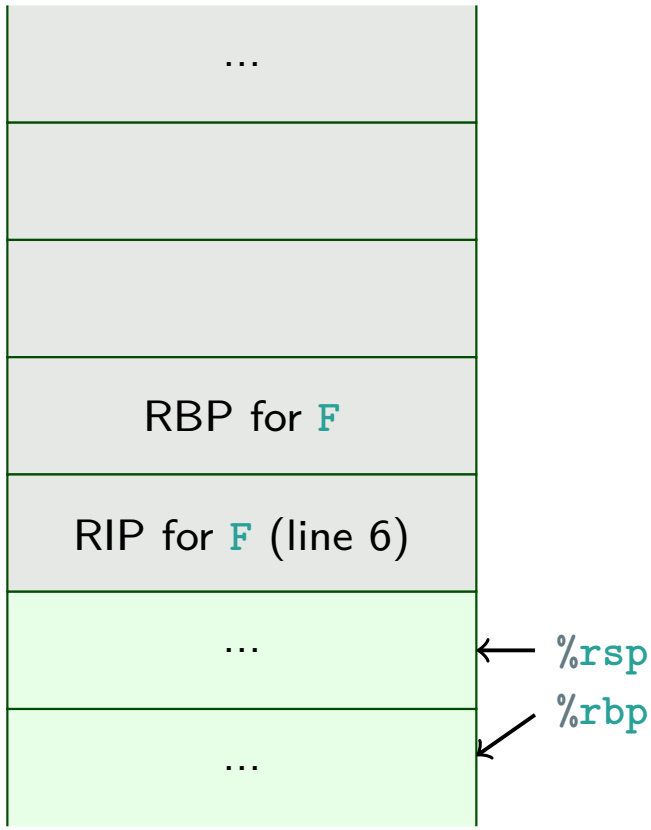
```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Example of Stack Frames, System V convention

This is the convention you will use.

```
1   F:
2       ...
3       movq $100, %rsi
4       movq $300, %rdi
5       call G
6       ...
7
8   G:
9       push %rbp
10      movq %rsp, %rbp
11      subq $16, %rsp
12
13      movq %rdi, %rax
14      addq %rsi, %rax
15
16      movq %rbp, %rsp
17      pop %rbp
18      ret
```



Function Prolog and Epilog

```
push %rbp
movq %rsp, %rbp
subq $xn, %rsp
```

is called the function prolog. Making space for locals is not always needed.

```
movq %rbp, %rsp
pop %rbp
```

is called the function epilog.

Handling Registers: caller/callee save

- What if a called function overwrites a register used by the caller?
- There may be many levels of nested function calls (also recursion).
- Let's use the stack, and a calling convention:
  - The callee knows that some registers must be restored before returning (callee save registers).
  - The caller knows that some registers may be overwritten by the callee (caller save registers).
- For x86-64 Linux:
  - Callee save registers: RBX, RBP, R12–R15
  - Caller save registers: the rest.
- For your programs: you decide.

Function Operations

Simplified:

- Calling: `call`
- Returning: `ret`
- Returning a value: RAX
- Passing arguments:
  - cdecl: stack
  - x86-64 Linux: RDI, RSI, RDX, RCX, R8, R9, stack
  - System call (64 bit): RDI, RSI, RDX, RCX, R8, R9
- Storing local variables when in a function: stack and RBP
- Handle registers, well-defined interference: caller/callee save

Just One More Thing . . .

- Registers for vector and float operations:
  - 128 bit width: XMM0–XMM15 (or XMM31)
  - 256 bit width: YMM0–YMM15 (or YMM31)
    extension of XMM registers
  - 512 bit width: ZMM0–ZMM31
    extension of YMM registers
- Alignment: not always needed, but performance depend on it. Generally, addresses should aligned depending on the data type.
- Endianness: how do we store a multi-byte value in memory?
  - Little endian: least significant byte first.
  - Big endian: most significant byte first.
  - Intel uses little endian.